

## 79 LL DMA Generic Driver

### 79.1 DMA Firmware driver registers structures

#### 79.1.1 LL\_DMA\_InitTypeDef

**LL\_DMA\_InitTypeDef** is defined in the stm32f4xx\_ll\_dma.h

Data Fields

- **uint32\_t PeriphOrM2MSrcAddress**
- **uint32\_t MemoryOrM2MDstAddress**
- **uint32\_t Direction**
- **uint32\_t Mode**
- **uint32\_t PeriphOrM2MSrcIncMode**
- **uint32\_t MemoryOrM2MDstIncMode**
- **uint32\_t PeriphOrM2MSrcDataSize**
- **uint32\_t MemoryOrM2MDstDataSize**
- **uint32\_t NbData**
- **uint32\_t Channel**
- **uint32\_t Priority**
- **uint32\_t FIFOMode**
- **uint32\_t FIFOThreshold**
- **uint32\_t MemBurst**
- **uint32\_t PeriphBurst**

Field Documentation

- **uint32\_t LL\_DMA\_InitTypeDef::PeriphOrM2MSrcAddress**  
Specifies the peripheral base address for DMA transfer or as Source base address in case of memory to memory transfer direction. This parameter must be a value between Min\_Data = 0 and Max\_Data = 0xFFFFFFFF.
- **uint32\_t LL\_DMA\_InitTypeDef::MemoryOrM2MDstAddress**  
Specifies the memory base address for DMA transfer or as Destination base address in case of memory to memory transfer direction. This parameter must be a value between Min\_Data = 0 and Max\_Data = 0xFFFFFFFF.
- **uint32\_t LL\_DMA\_InitTypeDef::Direction**  
Specifies if the data will be transferred from memory to peripheral, from memory to memory or from peripheral to memory. This parameter can be a value of [DMA\\_LL\\_EC\\_DIRECTION](#). This feature can be modified afterwards using unitary function [LL\\_DMA\\_SetDataTransferDirection\(\)](#).
- **uint32\_t LL\_DMA\_InitTypeDef::Mode**  
Specifies the normal or circular operation mode. This parameter can be a value of [DMA\\_LL\\_EC\\_MODE](#)  
**Note:**
  - The circular buffer mode cannot be used if the memory to memory data transfer direction is configured on the selected Stream
This feature can be modified afterwards using unitary function [LL\\_DMA\\_SetMode\(\)](#).
- **uint32\_t LL\_DMA\_InitTypeDef::PeriphOrM2MSrcIncMode**  
Specifies whether the Peripheral address or Source address in case of memory to memory transfer direction is incremented or not. This parameter can be a value of [DMA\\_LL\\_EC\\_PERIPH](#). This feature can be modified afterwards using unitary function [LL\\_DMA\\_SetPeriphIncMode\(\)](#).
- **uint32\_t LL\_DMA\_InitTypeDef::MemoryOrM2MDstIncMode**  
Specifies whether the Memory address or Destination address in case of memory to memory transfer direction is incremented or not. This parameter can be a value of [DMA\\_LL\\_EC\\_MEMORY](#). This feature can be modified afterwards using unitary function [LL\\_DMA\\_SetMemoryIncMode\(\)](#).

- **`uint32_t LL_DMA_InitTypeDef::PeriphOrM2MSrcDataSize`**  
Specifies the Peripheral data size alignment or Source data size alignment (byte, half word, word) in case of memory to memory transfer direction. This parameter can be a value of `DMA_LL_EC_PDATAALIGN`. This feature can be modified afterwards using unitary function `LL_DMA_SetPeriphSize()`.
- **`uint32_t LL_DMA_InitTypeDef::MemoryOrM2MDstDataSize`**  
Specifies the Memory data size alignment or Destination data size alignment (byte, half word, word) in case of memory to memory transfer direction. This parameter can be a value of `DMA_LL_EC_MDATAALIGN`. This feature can be modified afterwards using unitary function `LL_DMA_SetMemorySize()`.
- **`uint32_t LL_DMA_InitTypeDef::NbData`**  
Specifies the number of data to transfer, in data unit. The data unit is equal to the source buffer configuration set in `PeriphSize` or `MemorySize` parameters depending in the transfer direction. This parameter must be a value between `Min_Data = 0` and `Max_Data = 0x0000FFFF`. This feature can be modified afterwards using unitary function `LL_DMA_SetDataLength()`.
- **`uint32_t LL_DMA_InitTypeDef::Channel`**  
Specifies the peripheral channel. This parameter can be a value of `DMA_LL_EC_CHANNEL`. This feature can be modified afterwards using unitary function `LL_DMA_SetChannelSelection()`.
- **`uint32_t LL_DMA_InitTypeDef::Priority`**  
Specifies the channel priority level. This parameter can be a value of `DMA_LL_EC_PRIORITY`. This feature can be modified afterwards using unitary function `LL_DMA_SetStreamPriorityLevel()`.
- **`uint32_t LL_DMA_InitTypeDef::FIFOMode`**  
Specifies if the FIFO mode or Direct mode will be used for the specified stream. This parameter can be a value of `DMA_LL_FIFOMODE`.  
**Note:**
  - The Direct mode (FIFO mode disabled) cannot be used if the memory-to-memory data transfer is configured on the selected stream
This feature can be modified afterwards using unitary functions `LL_DMA_EnableFifoMode()` or `LL_DMA_EnableFifoMode()`.
- **`uint32_t LL_DMA_InitTypeDef::FIFOThreshold`**  
Specifies the FIFO threshold level. This parameter can be a value of `DMA_LL_EC_FIFOTHRESHOLD`. This feature can be modified afterwards using unitary function `LL_DMA_SetFIFOThreshold()`.
- **`uint32_t LL_DMA_InitTypeDef::MemBurst`**  
Specifies the Burst transfer configuration for the memory transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of `DMA_LL_EC_MBURST`.  
**Note:**
  - The burst mode is possible only if the address Increment mode is enabled.
This feature can be modified afterwards using unitary function `LL_DMA_SetMemoryBurstxfer()`.
- **`uint32_t LL_DMA_InitTypeDef::PeriphBurst`**  
Specifies the Burst transfer configuration for the peripheral transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of `DMA_LL_EC_PBURST`.  
**Note:**
  - The burst mode is possible only if the address Increment mode is enabled.
This feature can be modified afterwards using unitary function `LL_DMA_SetPeriphBurstxfer()`.

## 79.2 DMA Firmware driver API description

The following section lists the various functions of the DMA library.

### 79.2.1 Detailed description of functions

#### LL\_DMA\_EnableStream

##### Function name

```
__STATIC_INLINE void LL_DMA_EnableStream (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Enable DMA stream.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR EN LL\_DMA\_EnableStream

## LL\_DMA\_DisableStream

## Function name

```
__STATIC_INLINE void LL_DMA_DisableStream (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Disable DMA stream.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR EN LL\_DMA\_DisableStream

## LL\_DMA\_IsEnabledStream

## Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledStream (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Check if DMA stream is enabled or disabled.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- CR EN LL\_DMA\_IsEnabledStream

## LL\_DMA\_ConfigTransfer

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ConfigTransfer (DMA\_TypeDef \* DMAx, uint32\_t Stream, uint32\_t Configuration)**

### Function description

Configure all parameters linked to DMA transfer.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Configuration:** This parameter must be a combination of all the following values:
  - LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY or LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH or LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY
  - LL\_DMA\_MODE\_NORMAL or LL\_DMA\_MODE\_CIRCULAR or LL\_DMA\_MODE\_PFCTRL
  - LL\_DMA\_PERIPH\_INCREMENT or LL\_DMA\_PERIPH\_NOINCREMENT
  - LL\_DMA\_MEMORY\_INCREMENT or LL\_DMA\_MEMORY\_NOINCREMENT
  - LL\_DMA\_PDATAALIGN\_BYTE or LL\_DMA\_PDATAALIGN\_HALFWORD or LL\_DMA\_PDATAALIGN\_WORD
  - LL\_DMA\_MDATAALIGN\_BYTE or LL\_DMA\_MDATAALIGN\_HALFWORD or LL\_DMA\_MDATAALIGN\_WORD
  - LL\_DMA\_PRIORITY\_LOW or LL\_DMA\_PRIORITY\_MEDIUM or LL\_DMA\_PRIORITY\_HIGH or LL\_DMA\_PRIORITY\_VERYHIGH

## Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR DIR LL\_DMA\_ConfigTransfer
- CR CIRC LL\_DMA\_ConfigTransfer
- CR PINC LL\_DMA\_ConfigTransfer
- CR MINC LL\_DMA\_ConfigTransfer
- CR PSIZE LL\_DMA\_ConfigTransfer
- CR MSIZE LL\_DMA\_ConfigTransfer
- CR PL LL\_DMA\_ConfigTransfer
- CR PFCTRL LL\_DMA\_ConfigTransfer

#### LL\_DMA\_SetDataTransferDirection

##### Function name

```
__STATIC_INLINE void LL_DMA_SetDataTransferDirection (DMA_TypeDef * DMAx, uint32_t Stream,
uint32_t Direction)
```

##### Function description

Set Data transfer direction (read from peripheral or from memory).

##### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Direction:** This parameter can be one of the following values:
  - LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY

##### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR DIR LL\_DMA\_SetDataTransferDirection

#### LL\_DMA\_GetDataTransferDirection

##### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetDataTransferDirection (DMA_TypeDef * DMAx, uint32_t Stream)
```

##### Function description

Get Data transfer direction (read from peripheral or from memory).

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY

## Reference Manual to LL API cross reference:

- CR DIR LL\_DMA\_GetDataTransferDirection

## LL\_DMA\_SetMode

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_SetMode (DMA\_TypeDef \* DMAx, uint32\_t Stream, uint32\_t Mode)**

## Function description

Set DMA mode normal, circular or peripheral flow control.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Mode:** This parameter can be one of the following values:
  - LL\_DMA\_MODE\_NORMAL
  - LL\_DMA\_MODE\_CIRCULAR
  - LL\_DMA\_MODE\_PFCTRL

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR CIRC LL\_DMA\_SetMode
- CR PFCTRL LL\_DMA\_SetMode

## LL\_DMA\_GetMode

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get DMA mode normal, circular or peripheral flow control.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_MODE\_NORMAL
  - LL\_DMA\_MODE\_CIRCULAR
  - LL\_DMA\_MODE\_PFCTRL

### Reference Manual to LL API cross reference:

- CR CIRC LL\_DMA\_GetMode
- CR PFCTRL LL\_DMA\_GetMode

## LL\_DMA\_SetPeriphIncMode

### Function name

```
__STATIC_INLINE void LL_DMA_SetPeriphIncMode (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t IncrementMode)
```

### Function description

Set Peripheral increment mode.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **IncrementMode:** This parameter can be one of the following values:
  - LL\_DMA\_PERIPH\_NOINCREMENT
  - LL\_DMA\_PERIPH\_INCREMENT

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR PINC LL\_DMA\_SetPeriphIncMode

#### LL\_DMA\_GetPeriphIncMode

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetPeriphIncMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Get Peripheral increment mode.

#### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

#### Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_PERIPH\_NOINCREMENT
  - LL\_DMA\_PERIPH\_INCREMENT

#### Reference Manual to LL API cross reference:

- CR PINC LL\_DMA\_GetPeriphIncMode

#### LL\_DMA\_SetMemoryIncMode

#### Function name

```
__STATIC_INLINE void LL_DMA_SetMemoryIncMode (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t IncrementMode)
```

#### Function description

Set Memory increment mode.



## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **IncrementMode:** This parameter can be one of the following values:
  - LL\_DMA\_MEMORY\_NOINCREMENT
  - LL\_DMA\_MEMORY\_INCREMENT

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR MINC LL\_DMA\_SetMemoryIncMode

### LL\_DMA\_GetMemoryIncMode

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_GetMemoryIncMode (DMA\_TypeDef \* DMAx, uint32\_t Stream)**

## Function description

Get Memory increment mode.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_MEMORY\_NOINCREMENT
  - LL\_DMA\_MEMORY\_INCREMENT

## Reference Manual to LL API cross reference:

- CR MINC LL\_DMA\_GetMemoryIncMode

### LL\_DMA\_SetPeriphSize

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_SetPeriphSize (DMA\_TypeDef \* DMAx, uint32\_t Stream, uint32\_t Size)**

## Function description

Set Peripheral size.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Size:** This parameter can be one of the following values:
  - LL\_DMA\_PDATAALIGN\_BYTE
  - LL\_DMA\_PDATAALIGN\_HALFWORD
  - LL\_DMA\_PDATAALIGN\_WORD

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR PSIZE LL\_DMA\_SetPeriphSize

## LL\_DMA\_GetPeriphSize

## Function name

`__STATIC_INLINE uint32_t LL_DMA_GetPeriphSize (DMA_TypeDef * DMAx, uint32_t Stream)`

## Function description

Get Peripheral size.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_PDATAALIGN\_BYTE
  - LL\_DMA\_PDATAALIGN\_HALFWORD
  - LL\_DMA\_PDATAALIGN\_WORD

## Reference Manual to LL API cross reference:

- CR PSIZE LL\_DMA\_GetPeriphSize

## LL\_DMA\_SetMemorySize

### Function name

```
__STATIC_INLINE void LL_DMA_SetMemorySize (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Size)
```

### Function description

Set Memory size.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Size:** This parameter can be one of the following values:
  - LL\_DMA\_MDATAALIGN\_BYTE
  - LL\_DMA\_MDATAALIGN\_HALFWORD
  - LL\_DMA\_MDATAALIGN\_WORD

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR MSIZE LL\_DMA\_SetMemorySize

## LL\_DMA\_GetMemorySize

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMemorySize (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get Memory size.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_MDATAALIGN\_BYTE
  - LL\_DMA\_MDATAALIGN\_HALFWORD
  - LL\_DMA\_MDATAALIGN\_WORD

## Reference Manual to LL API cross reference:

- CR MSIZE LL\_DMA\_GetMemorySize

## LL\_DMA\_SetIncOffsetSize

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_SetIncOffsetSize (DMA\_TypeDef \* DMAx, uint32\_t Stream, uint32\_t OffsetSize)**

## Function description

Set Peripheral increment offset size.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **OffsetSize:** This parameter can be one of the following values:
  - LL\_DMA\_OFFSETSIZE\_PSIZE
  - LL\_DMA\_OFFSETSIZE\_FIXEDTO4

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR PINCOS LL\_DMA\_SetIncOffsetSize

## LL\_DMA\_GetIncOffsetSize

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_GetIncOffsetSize (DMA\_TypeDef \* DMAx, uint32\_t Stream)**

## Function description

Get Peripheral increment offset size.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_OFFSETSIZE\_PSIZE
  - LL\_DMA\_OFFSETSIZE\_FIXEDTO4

## Reference Manual to LL API cross reference:

- CR PINCOS LL\_DMA\_GetIncOffsetSize

## LL\_DMA\_SetStreamPriorityLevel

## Function name

```
__STATIC_INLINE void LL_DMA_SetStreamPriorityLevel (DMA_TypeDef * DMAx, uint32_t Stream,
uint32_t Priority)
```

## Function description

Set Stream priority level.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Priority:** This parameter can be one of the following values:
  - LL\_DMA\_PRIORITY\_LOW
  - LL\_DMA\_PRIORITY\_MEDIUM
  - LL\_DMA\_PRIORITY\_HIGH
  - LL\_DMA\_PRIORITY\_VERYHIGH

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR PL LL\_DMA\_SetStreamPriorityLevel

## LL\_DMA\_GetStreamPriorityLevel

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetStreamPriorityLevel (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get Stream priority level.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_PRIORITY\_LOW
  - LL\_DMA\_PRIORITY\_MEDIUM
  - LL\_DMA\_PRIORITY\_HIGH
  - LL\_DMA\_PRIORITY\_VERYHIGH

### Reference Manual to LL API cross reference:

- CR PL LL\_DMA\_GetStreamPriorityLevel

## LL\_DMA\_SetDataLength

### Function name

```
__STATIC_INLINE void LL_DMA_SetDataLength (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t NbData)
```

### Function description

Set Number of data to transfer.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **NbData:** Between 0 to 0xFFFFFFFF

## Return values

- **None:**

## Notes

- This action has no effect if stream is enabled.

## Reference Manual to LL API cross reference:

- NDTR NDT LL\_DMA\_SetDataLength

## LL\_DMA\_GetDataLength

## Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetDataLength (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Get Number of data to transfer.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Between:** 0 to 0xFFFFFFFF

## Notes

- Once the stream is enabled, the return value indicate the remaining bytes to be transmitted.

## Reference Manual to LL API cross reference:

- NDTR NDT LL\_DMA\_GetDataLength

## LL\_DMA\_SetChannelSelection

## Function name

```
__STATIC_INLINE void LL_DMA_SetChannelSelection (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Channel)
```

## Function description

Select Channel number associated to the Stream.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Channel:** This parameter can be one of the following values:
  - LL\_DMA\_CHANNEL\_0
  - LL\_DMA\_CHANNEL\_1
  - LL\_DMA\_CHANNEL\_2
  - LL\_DMA\_CHANNEL\_3
  - LL\_DMA\_CHANNEL\_4
  - LL\_DMA\_CHANNEL\_5
  - LL\_DMA\_CHANNEL\_6
  - LL\_DMA\_CHANNEL\_7

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR CHSEL LL\_DMA\_SetChannelSelection

## LL\_DMA\_GetChannelSelection

## Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetChannelSelection (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Get the Channel number associated to the Stream.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7



## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_CHANNEL\_0
  - LL\_DMA\_CHANNEL\_1
  - LL\_DMA\_CHANNEL\_2
  - LL\_DMA\_CHANNEL\_3
  - LL\_DMA\_CHANNEL\_4
  - LL\_DMA\_CHANNEL\_5
  - LL\_DMA\_CHANNEL\_6
  - LL\_DMA\_CHANNEL\_7

## Reference Manual to LL API cross reference:

- CR CHSEL LL\_DMA\_GetChannelSelection

## LL\_DMA\_SetMemoryBurstxfer

### Function name

```
__STATIC_INLINE void LL_DMA_SetMemoryBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Mburst)
```

### Function description

Set Memory burst transfer configuration.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Mburst:** This parameter can be one of the following values:
  - LL\_DMA\_MBURST\_SINGLE
  - LL\_DMA\_MBURST\_INC4
  - LL\_DMA\_MBURST\_INC8
  - LL\_DMA\_MBURST\_INC16

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR MBURST LL\_DMA\_SetMemoryBurstxfer

## LL\_DMA\_GetMemoryBurstxfer

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMemoryBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get Memory burst transfer configuration.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_MBURST\_SINGLE
  - LL\_DMA\_MBURST\_INC4
  - LL\_DMA\_MBURST\_INC8
  - LL\_DMA\_MBURST\_INC16

## Reference Manual to LL API cross reference:

- CR MBURST LL\_DMA\_GetMemoryBurstxfer

## LL\_DMA\_SetPeriphBurstxfer

## Function name

```
__STATIC_INLINE void LL_DMA_SetPeriphBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Pburst)
```

## Function description

Set Peripheral burst transfer configuration.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Pburst:** This parameter can be one of the following values:
  - LL\_DMA\_PBURST\_SINGLE
  - LL\_DMA\_PBURST\_INC4
  - LL\_DMA\_PBURST\_INC8
  - LL\_DMA\_PBURST\_INC16

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR PBURST LL\_DMA\_SetPeriphBurstxfer

## LL\_DMA\_GetPeriphBurstxfer

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetPeriphBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get Peripheral burst transfer configuration.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_PBURST\_SINGLE
  - LL\_DMA\_PBURST\_INC4
  - LL\_DMA\_PBURST\_INC8
  - LL\_DMA\_PBURST\_INC16

### Reference Manual to LL API cross reference:

- CR PBURST LL\_DMA\_GetPeriphBurstxfer

## LL\_DMA\_SetCurrentTargetMem

### Function name

```
__STATIC_INLINE void LL_DMA_SetCurrentTargetMem (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t CurrentMemory)
```

### Function description

Set Current target (only in double buffer mode) to Memory 1 or Memory 0.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **CurrentMemory:** This parameter can be one of the following values:
  - LL\_DMA\_CURRENTTARGETMEM0
  - LL\_DMA\_CURRENTTARGETMEM1

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CR CT LL\_DMA\_SetCurrentTargetMem

## LL\_DMA\_GetCurrentTargetMem

## Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetCurrentTargetMem (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Set Current target (only in double buffer mode) to Memory 1 or Memory 0.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_CURRENTTARGETMEM0
  - LL\_DMA\_CURRENTTARGETMEM1

## Reference Manual to LL API cross reference:

- CR CT LL\_DMA\_GetCurrentTargetMem

## LL\_DMA\_EnableDoubleBufferMode

## Function name

```
__STATIC_INLINE void LL_DMA_EnableDoubleBufferMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Enable the double buffer mode.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR DBM LL\_DMA\_EnableDoubleBufferMode

#### LL\_DMA\_DisableDoubleBufferMode

#### Function name

```
__STATIC_INLINE void LL_DMA_DisableDoubleBufferMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Disable the double buffer mode.

#### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR DBM LL\_DMA\_DisableDoubleBufferMode

#### LL\_DMA\_GetFIFOStatus

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetFIFOStatus (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Get FIFO status.

#### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_FIFOSTATUS\_0\_25
  - LL\_DMA\_FIFOSTATUS\_25\_50
  - LL\_DMA\_FIFOSTATUS\_50\_75
  - LL\_DMA\_FIFOSTATUS\_75\_100
  - LL\_DMA\_FIFOSTATUS\_EMPTY
  - LL\_DMA\_FIFOSTATUS\_FULL

### Reference Manual to LL API cross reference:

- FCR FS LL\_DMA\_GetFIFOStatus

### LL\_DMA\_DisableFifoMode

#### Function name

```
__STATIC_INLINE void LL_DMA_DisableFifoMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Disable Fifo mode.

#### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- FCR DMDIS LL\_DMA\_DisableFifoMode

### LL\_DMA\_EnableFifoMode

#### Function name

```
__STATIC_INLINE void LL_DMA_EnableFifoMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Enable Fifo mode.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- FCR DMDIS LL\_DMA\_EnableFifoMode

## LL\_DMA\_SetFIFOThreshold

## Function name

```
__STATIC_INLINE void LL_DMA_SetFIFOThreshold (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Threshold)
```

## Function description

Select FIFO threshold.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Threshold:** This parameter can be one of the following values:
  - LL\_DMA\_FIFOTHRESHOLD\_1\_4
  - LL\_DMA\_FIFOTHRESHOLD\_1\_2
  - LL\_DMA\_FIFOTHRESHOLD\_3\_4
  - LL\_DMA\_FIFOTHRESHOLD\_FULL

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- FCR FTH LL\_DMA\_GetFIFOThreshold

## LL\_DMA\_GetFIFOThreshold

## Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetFIFOThreshold (DMA_TypeDef * DMAx, uint32_t Stream)
```

## Function description

Get FIFO threshold.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_DMA\_FIFOTHRESHOLD\_1\_4
  - LL\_DMA\_FIFOTHRESHOLD\_1\_2
  - LL\_DMA\_FIFOTHRESHOLD\_3\_4
  - LL\_DMA\_FIFOTHRESHOLD\_FULL

## Reference Manual to LL API cross reference:

- FCR FTH LL\_DMA\_GetFIFOThreshold

## LL\_DMA\_ConfigFifo

## Function name

```
__STATIC_INLINE void LL_DMA_ConfigFifo (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t FifoMode,
uint32_t FifoThreshold)
```

## Function description

Configure the FIFO .

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **FifoMode:** This parameter can be one of the following values:
  - LL\_DMA\_FIFOMODE\_ENABLE
  - LL\_DMA\_FIFOMODE\_DISABLE
- **FifoThreshold:** This parameter can be one of the following values:
  - LL\_DMA\_FIFOTHRESHOLD\_1\_4
  - LL\_DMA\_FIFOTHRESHOLD\_1\_2
  - LL\_DMA\_FIFOTHRESHOLD\_3\_4
  - LL\_DMA\_FIFOTHRESHOLD\_FULL



## Return values

- **None:**

## Reference Manual to LL API cross reference:

- FCR FTH LL\_DMA\_ConfigFifo
- FCR DMDIS LL\_DMA\_ConfigFifo

### LL\_DMA\_ConfigAddresses

## Function name

```
__STATIC_INLINE void LL_DMA_ConfigAddresses (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t SrcAddress, uint32_t DstAddress, uint32_t Direction)
```

## Function description

Configure the Source and Destination addresses.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **SrcAddress:** Between 0 to 0xFFFFFFFF
- **DstAddress:** Between 0 to 0xFFFFFFFF
- **Direction:** This parameter can be one of the following values:
  - LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH
  - LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY

## Return values

- **None:**

## Notes

- This API must not be called when the DMA stream is enabled.

## Reference Manual to LL API cross reference:

- M0AR M0A LL\_DMA\_ConfigAddresses
- PAR PA LL\_DMA\_ConfigAddresses

### LL\_DMA\_SetMemoryAddress

## Function name

```
__STATIC_INLINE void LL_DMA_SetMemoryAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)
```

## Function description

Set the Memory address.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **MemoryAddress:** Between 0 to 0xFFFFFFFF

## Return values

- **None:**

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY or LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH only.
- This API must not be called when the DMA channel is enabled.

## Reference Manual to LL API cross reference:

- M0AR M0A LL\_DMA\_SetMemoryAddress

## LL\_DMA\_SetPeriphAddress

## Function name

```
__STATIC_INLINE void LL_DMA_SetPeriphAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t PeriphAddress)
```

## Function description

Set the Peripheral address.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **PeriphAddress:** Between 0 to 0xFFFFFFFF

## Return values

- **None:**

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY or LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH only.
- This API must not be called when the DMA channel is enabled.

#### Reference Manual to LL API cross reference:

- PAR PA LL\_DMA\_SetPeriphAddress

#### LL\_DMA\_GetMemoryAddress

##### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMemoryAddress (DMA_TypeDef * DMAx, uint32_t Stream)
```

##### Function description

Get the Memory address.

##### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

##### Return values

- **Between:** 0 to 0xFFFFFFFF

##### Notes

- Interface used for direction LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY or LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH only.

#### Reference Manual to LL API cross reference:

- M0AR M0A LL\_DMA\_GetMemoryAddress

#### LL\_DMA\_GetPeriphAddress

##### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetPeriphAddress (DMA_TypeDef * DMAx, uint32_t Stream)
```

##### Function description

Get the Peripheral address.

##### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

##### Return values

- **Between:** 0 to 0xFFFFFFFF

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY or LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH only.

## Reference Manual to LL API cross reference:

- PAR PA LL\_DMA\_GetPeriphAddress

### LL\_DMA\_SetM2MSrcAddress

## Function name

```
__STATIC_INLINE void LL_DMA_SetM2MSrcAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)
```

## Function description

Set the Memory to Memory Source address.

## Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- MemoryAddress:** Between 0 to 0xFFFFFFFF

## Return values

- None:**

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY only.
- This API must not be called when the DMA channel is enabled.

## Reference Manual to LL API cross reference:

- PAR PA LL\_DMA\_SetM2MSrcAddress

### LL\_DMA\_SetM2MDstAddress

## Function name

```
__STATIC_INLINE void LL_DMA_SetM2MDstAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)
```

## Function description

Set the Memory to Memory Destination address.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **MemoryAddress:** Between 0 to 0xFFFFFFFF

## Return values

- **None:**

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY only.
- This API must not be called when the DMA channel is enabled.

## Reference Manual to LL API cross reference:

- M0AR M0A LL\_DMA\_SetM2MDstAddress

## LL\_DMA\_GetM2MSrcAddress

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_GetM2MSrcAddress (DMA\_TypeDef \* DMAx, uint32\_t Stream)**

## Function description

Get the Memory to Memory Source address.

## Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

## Return values

- **Between:** 0 to 0xFFFFFFFF

## Notes

- Interface used for direction LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY only.

## Reference Manual to LL API cross reference:

- PAR PA LL\_DMA\_GetM2MSrcAddress

## LL\_DMA\_GetM2MDstAddress

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetM2MDstAddress (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Get the Memory to Memory Destination address.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **Between:** 0 to 0xFFFFFFFF

### Notes

- Interface used for direction LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY only.

### Reference Manual to LL API cross reference:

- M0AR M0A LL\_DMA\_GetM2MDstAddress

## LL\_DMA\_SetMemory1Address

### Function name

```
__STATIC_INLINE void LL_DMA_SetMemory1Address (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Address)
```

### Function description

Set Memory 1 address (used in case of Double buffer mode).

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **Address:** Between 0 to 0xFFFFFFFF

### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- M1AR M1A LL\_DMA\_SetMemory1Address

#### LL\_DMA\_GetMemory1Address

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMemory1Address (DMA_TypeDef * DMAx, uint32_t Stream)
```

#### Function description

Get Memory 1 address (used in case of Double buffer mode).

#### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

#### Return values

- **Between:** 0 to 0xFFFFFFFF

#### Reference Manual to LL API cross reference:

- M1AR M1A LL\_DMA\_GetMemory1Address

#### LL\_DMA\_IsActiveFlag\_HT0

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT0 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 0 half transfer flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR HTIF0 LL\_DMA\_IsActiveFlag\_HT0

#### LL\_DMA\_IsActiveFlag\_HT1

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT1 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 1 half transfer flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR HTIF1 LL\_DMA\_IsActiveFlag\_HT1

LL\_DMA\_IsActiveFlag\_HT2

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_HT2 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 2 half transfer flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR HTIF2 LL\_DMA\_IsActiveFlag\_HT2

LL\_DMA\_IsActiveFlag\_HT3

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_HT3 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 3 half transfer flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR HTIF3 LL\_DMA\_IsActiveFlag\_HT3

LL\_DMA\_IsActiveFlag\_HT4

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_HT4 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 4 half transfer flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR HTIF4 LL\_DMA\_IsActiveFlag\_HT4



## LL\_DMA\_IsActiveFlag\_HT5

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT5 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 5 half transfer flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR HTIF0 LL\_DMA\_IsActiveFlag\_HT5

## LL\_DMA\_IsActiveFlag\_HT6

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT6 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 6 half transfer flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR HTIF6 LL\_DMA\_IsActiveFlag\_HT6

## LL\_DMA\_IsActiveFlag\_HT7

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT7 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 7 half transfer flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR HTIF7 LL\_DMA\_IsActiveFlag\_HT7

## LL\_DMA\_IsActiveFlag\_TC0

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC0 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 0 transfer complete flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- LISR TCIF0 LL\_DMA\_IsActiveFlag\_TC0

**LL\_DMA\_IsActiveFlag\_TC1**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC1 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 1 transfer complete flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- LISR TCIF1 LL\_DMA\_IsActiveFlag\_TC1

**LL\_DMA\_IsActiveFlag\_TC2**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC2 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 2 transfer complete flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- LISR TCIF2 LL\_DMA\_IsActiveFlag\_TC2

**LL\_DMA\_IsActiveFlag\_TC3**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC3 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 3 transfer complete flag.

### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR TCIF3 LL\_DMA\_IsActiveFlag\_TC3

**LL\_DMA\_IsActiveFlag\_TC4**

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC4 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 4 transfer complete flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TCIF4 LL\_DMA\_IsActiveFlag\_TC4

**LL\_DMA\_IsActiveFlag\_TC5**

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC5 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 5 transfer complete flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TCIF0 LL\_DMA\_IsActiveFlag\_TC5

**LL\_DMA\_IsActiveFlag\_TC6**

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TC6 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 6 transfer complete flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TCIF6 LL\_DMA\_IsActiveFlag\_TC6

### LL\_DMA\_IsActiveFlag\_TC7

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC7 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 7 transfer complete flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TCIF7 LL\_DMA\_IsActiveFlag\_TC7

### LL\_DMA\_IsActiveFlag\_TE0

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE0 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 0 transfer error flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR TEIF0 LL\_DMA\_IsActiveFlag\_TE0

### LL\_DMA\_IsActiveFlag\_TE1

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE1 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 1 transfer error flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR TEIF1 LL\_DMA\_IsActiveFlag\_TE1

### LL\_DMA\_IsActiveFlag\_TE2

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE2 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 2 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- LISR TEIF2 LL\_DMA\_IsActiveFlag\_TE2

**LL\_DMA\_IsActiveFlag\_TE3**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TE3 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 3 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- LISR TEIF3 LL\_DMA\_IsActiveFlag\_TE3

**LL\_DMA\_IsActiveFlag\_TE4**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TE4 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 4 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR TEIF4 LL\_DMA\_IsActiveFlag\_TE4

**LL\_DMA\_IsActiveFlag\_TE5**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TE5 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 5 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TEIF0 LL\_DMA\_IsActiveFlag\_TE5

LL\_DMA\_IsActiveFlag\_TE6

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TE6 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 6 transfer error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TEIF6 LL\_DMA\_IsActiveFlag\_TE6

LL\_DMA\_IsActiveFlag\_TE7

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_TE7 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 7 transfer error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR TEIF7 LL\_DMA\_IsActiveFlag\_TE7

LL\_DMA\_IsActiveFlag\_DME0

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_DME0 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 0 direct mode error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR DMEIF0 LL\_DMA\_IsActiveFlag\_DME0

### LL\_DMA\_IsActiveFlag\_DME1

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME1 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 1 direct mode error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR DMEIF1 LL\_DMA\_IsActiveFlag\_DME1

### LL\_DMA\_IsActiveFlag\_DME2

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME2 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 2 direct mode error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR DMEIF2 LL\_DMA\_IsActiveFlag\_DME2

### LL\_DMA\_IsActiveFlag\_DME3

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME3 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 3 direct mode error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR DMEIF3 LL\_DMA\_IsActiveFlag\_DME3

### LL\_DMA\_IsActiveFlag\_DME4

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME4 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 4 direct mode error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR DMEIF4 LL\_DMA\_IsActiveFlag\_DME4

**LL\_DMA\_IsActiveFlag\_DME5**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_DME5 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 5 direct mode error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR DMEIF0 LL\_DMA\_IsActiveFlag\_DME5

**LL\_DMA\_IsActiveFlag\_DME6**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_DME6 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 6 direct mode error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **State**: of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR DMEIF6 LL\_DMA\_IsActiveFlag\_DME6

**LL\_DMA\_IsActiveFlag\_DME7**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_DME7 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 7 direct mode error flag.

### Parameters

- **DMAx**: DMAx Instance



#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR DMEIF7 LL\_DMA\_IsActiveFlag\_DME7

LL\_DMA\_IsActiveFlag\_FE0

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_FE0 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 0 FIFO error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR FEIF0 LL\_DMA\_IsActiveFlag\_FE0

LL\_DMA\_IsActiveFlag\_FE1

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_FE1 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 1 FIFO error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR FEIF1 LL\_DMA\_IsActiveFlag\_FE1

LL\_DMA\_IsActiveFlag\_FE2

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_FE2 (DMA\_TypeDef \* DMAx)**

#### Function description

Get Stream 2 FIFO error flag.

#### Parameters

- **DMAx:** DMAx Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR FEIF2 LL\_DMA\_IsActiveFlag\_FE2

### LL\_DMA\_IsActiveFlag\_FE3

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE3 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 3 FIFO error flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- LISR FEIF3 LL\_DMA\_IsActiveFlag\_FE3

### LL\_DMA\_IsActiveFlag\_FE4

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE4 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 4 FIFO error flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR FEIF4 LL\_DMA\_IsActiveFlag\_FE4

### LL\_DMA\_IsActiveFlag\_FE5

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE5 (DMA_TypeDef * DMAx)
```

#### Function description

Get Stream 5 FIFO error flag.

#### Parameters

- **DMAx**: DMAx Instance

#### Return values

- **State**: of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- HISR FEIF0 LL\_DMA\_IsActiveFlag\_FE5

### LL\_DMA\_IsActiveFlag\_FE6

#### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE6 (DMA_TypeDef * DMAx)
```

### Function description

Get Stream 6 FIFO error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR FEIF6 LL\_DMA\_IsActiveFlag\_FE6

LL\_DMA\_IsActiveFlag\_FE7

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_DMA\_IsActiveFlag\_FE7 (DMA\_TypeDef \* DMAx)**

### Function description

Get Stream 7 FIFO error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- HISR FEIF7 LL\_DMA\_IsActiveFlag\_FE7

LL\_DMA\_ClearFlag\_HT0

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_HT0 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 0 half transfer flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- LIFCR CHTIF0 LL\_DMA\_ClearFlag\_HT0

LL\_DMA\_ClearFlag\_HT1

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_HT1 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 1 half transfer flag.

### Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CHTIF1 LL\_DMA\_ClearFlag\_HT1

## LL\_DMA\_ClearFlag\_HT2

## Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT2 (DMA_TypeDef * DMAx)
```

## Function description

Clear Stream 2 half transfer flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CHTIF2 LL\_DMA\_ClearFlag\_HT2

## LL\_DMA\_ClearFlag\_HT3

## Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT3 (DMA_TypeDef * DMAx)
```

## Function description

Clear Stream 3 half transfer flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CHTIF3 LL\_DMA\_ClearFlag\_HT3

## LL\_DMA\_ClearFlag\_HT4

## Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT4 (DMA_TypeDef * DMAx)
```

## Function description

Clear Stream 4 half transfer flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- HIFCR CHTIF4 LL\_DMA\_ClearFlag\_HT4

## LL\_DMA\_ClearFlag\_HT5

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT5 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 5 half transfer flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CHTIF5 LL\_DMA\_ClearFlag\_HT5

## LL\_DMA\_ClearFlag\_HT6

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT6 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 6 half transfer flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CHTIF6 LL\_DMA\_ClearFlag\_HT6

## LL\_DMA\_ClearFlag\_HT7

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT7 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 7 half transfer flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CHTIF7 LL\_DMA\_ClearFlag\_HT7

## LL\_DMA\_ClearFlag\_TC0

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TC0 (DMA_TypeDef * DMAx)
```

## Function description

Clear Stream 0 transfer complete flag.

## Parameters

- **DMAx**: DMAx Instance

## Return values

- **None**:

## Reference Manual to LL API cross reference:

- LIFCR CTCIF0 LL\_DMA\_ClearFlag\_TC0

**LL\_DMA\_ClearFlag\_TC1**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC1 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 1 transfer complete flag.

## Parameters

- **DMAx**: DMAx Instance

## Return values

- **None**:

## Reference Manual to LL API cross reference:

- LIFCR CTCIF1 LL\_DMA\_ClearFlag\_TC1

**LL\_DMA\_ClearFlag\_TC2**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC2 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 2 transfer complete flag.

## Parameters

- **DMAx**: DMAx Instance

## Return values

- **None**:

## Reference Manual to LL API cross reference:

- LIFCR CTCIF2 LL\_DMA\_ClearFlag\_TC2

**LL\_DMA\_ClearFlag\_TC3**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC3 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 3 transfer complete flag.

## Parameters

- **DMAx**: DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- LIFCR CTCIF3 LL\_DMA\_ClearFlag\_TC3

**LL\_DMA\_ClearFlag\_TC4****Function name**

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC4 (DMA\_TypeDef \* DMAx)**

**Function description**

Clear Stream 4 transfer complete flag.

**Parameters**

- **DMAx:** DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- HIFCR CTCIF4 LL\_DMA\_ClearFlag\_TC4

**LL\_DMA\_ClearFlag\_TC5****Function name**

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC5 (DMA\_TypeDef \* DMAx)**

**Function description**

Clear Stream 5 transfer complete flag.

**Parameters**

- **DMAx:** DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- HIFCR CTCIF5 LL\_DMA\_ClearFlag\_TC5

**LL\_DMA\_ClearFlag\_TC6****Function name**

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TC6 (DMA\_TypeDef \* DMAx)**

**Function description**

Clear Stream 6 transfer complete flag.

**Parameters**

- **DMAx:** DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- HIFCR CTCIF6 LL\_DMA\_ClearFlag\_TC6

## LL\_DMA\_ClearFlag\_TC7

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TC7 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 7 transfer complete flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CTCIF7 LL\_DMA\_ClearFlag\_TC7

## LL\_DMA\_ClearFlag\_TE0

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE0 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 0 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- LIFCR CTEIF0 LL\_DMA\_ClearFlag\_TE0

## LL\_DMA\_ClearFlag\_TE1

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE1 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 1 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- LIFCR CTEIF1 LL\_DMA\_ClearFlag\_TE1

## LL\_DMA\_ClearFlag\_TE2

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE2 (DMA_TypeDef * DMAx)
```



### Function description

Clear Stream 2 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None**:

### Reference Manual to LL API cross reference:

- LIFCR CTEIF2 LL\_DMA\_ClearFlag\_TE2

**LL\_DMA\_ClearFlag\_TE3**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TE3 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 3 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None**:

### Reference Manual to LL API cross reference:

- LIFCR CTEIF3 LL\_DMA\_ClearFlag\_TE3

**LL\_DMA\_ClearFlag\_TE4**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TE4 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 4 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None**:

### Reference Manual to LL API cross reference:

- HIFCR CTEIF4 LL\_DMA\_ClearFlag\_TE4

**LL\_DMA\_ClearFlag\_TE5**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TE5 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 5 transfer error flag.

### Parameters

- **DMAx**: DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- HIFCR CTEIF5 LL\_DMA\_ClearFlag\_TE5

**LL\_DMA\_ClearFlag\_TE6**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TE6 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 6 transfer error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- HIFCR CTEIF6 LL\_DMA\_ClearFlag\_TE6

**LL\_DMA\_ClearFlag\_TE7**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_TE7 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 7 transfer error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- HIFCR CTEIF7 LL\_DMA\_ClearFlag\_TE7

**LL\_DMA\_ClearFlag\_DME0**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_DME0 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 0 direct mode error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CDMEIF0 LL\_DMA\_ClearFlag\_DME0

**LL\_DMA\_ClearFlag\_DME1****Function name**

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME1 (DMA_TypeDef * DMAx)
```

**Function description**

Clear Stream 1 direct mode error flag.

**Parameters**

- **DMAx**: DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- LIFCR CDMEIF1 LL\_DMA\_ClearFlag\_DME1

**LL\_DMA\_ClearFlag\_DME2****Function name**

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME2 (DMA_TypeDef * DMAx)
```

**Function description**

Clear Stream 2 direct mode error flag.

**Parameters**

- **DMAx**: DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- LIFCR CDMEIF2 LL\_DMA\_ClearFlag\_DME2

**LL\_DMA\_ClearFlag\_DME3****Function name**

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME3 (DMA_TypeDef * DMAx)
```

**Function description**

Clear Stream 3 direct mode error flag.

**Parameters**

- **DMAx**: DMAx Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- LIFCR CDMEIF3 LL\_DMA\_ClearFlag\_DME3

**LL\_DMA\_ClearFlag\_DME4****Function name**

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME4 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 4 direct mode error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CDMEIF4 LL\_DMA\_ClearFlag\_DME4

**LL\_DMA\_ClearFlag\_DME5**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_DME5 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 5 direct mode error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CDMEIF5 LL\_DMA\_ClearFlag\_DME5

**LL\_DMA\_ClearFlag\_DME6**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_DME6 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 6 direct mode error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CDMEIF6 LL\_DMA\_ClearFlag\_DME6

**LL\_DMA\_ClearFlag\_DME7**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_DME7 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 7 direct mode error flag.

### Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- HIFCR CDMEIF7 LL\_DMA\_ClearFlag\_DME7

**LL\_DMA\_ClearFlag\_FE0**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_FE0 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 0 FIFO error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CFEIF0 LL\_DMA\_ClearFlag\_FE0

**LL\_DMA\_ClearFlag\_FE1**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_FE1 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 1 FIFO error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CFEIF1 LL\_DMA\_ClearFlag\_FE1

**LL\_DMA\_ClearFlag\_FE2**

## Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_FE2 (DMA\_TypeDef \* DMAx)**

## Function description

Clear Stream 2 FIFO error flag.

## Parameters

- **DMAx:** DMAx Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- LIFCR CFEIF2 LL\_DMA\_ClearFlag\_FE2

## LL\_DMA\_ClearFlag\_FE3

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE3 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 3 FIFO error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- LIFCR CFEIF3 LL\_DMA\_ClearFlag\_FE3

## LL\_DMA\_ClearFlag\_FE4

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE4 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 4 FIFO error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CFEIF4 LL\_DMA\_ClearFlag\_FE4

## LL\_DMA\_ClearFlag\_FE5

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE5 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 5 FIFO error flag.

### Parameters

- **DMAx**: DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CFEIF5 LL\_DMA\_ClearFlag\_FE5

## LL\_DMA\_ClearFlag\_FE6

### Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE6 (DMA_TypeDef * DMAx)
```

### Function description

Clear Stream 6 FIFO error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CFEIF6 LL\_DMA\_ClearFlag\_FE6

**LL\_DMA\_ClearFlag\_FE7**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_ClearFlag\_FE7 (DMA\_TypeDef \* DMAx)**

### Function description

Clear Stream 7 FIFO error flag.

### Parameters

- **DMAx:** DMAx Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- HIFCR CFEIF7 LL\_DMA\_ClearFlag\_FE7

**LL\_DMA\_EnableIT\_HT**

### Function name

**\_\_STATIC\_INLINE void LL\_DMA\_EnableIT\_HT (DMA\_TypeDef \* DMAx, uint32\_t Stream)**

### Function description

Enable Half transfer interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR HTIE LL\_DMA\_EnableIT\_HT

## LL\_DMA\_EnableIT\_TE

### Function name

```
__STATIC_INLINE void LL_DMA_EnableIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Enable Transfer error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR TEIE LL\_DMA\_EnableIT\_TE

## LL\_DMA\_EnableIT\_TC

### Function name

```
__STATIC_INLINE void LL_DMA_EnableIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Enable Transfer complete interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR TCIE LL\_DMA\_EnableIT\_TC



## LL\_DMA\_EnableIT\_DME

### Function name

```
__STATIC_INLINE void LL_DMA_EnableIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Enable Direct mode error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR DMEIE LL\_DMA\_EnableIT\_DME

## LL\_DMA\_EnableIT\_FE

### Function name

```
__STATIC_INLINE void LL_DMA_EnableIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Enable FIFO error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- FCR FEIE LL\_DMA\_EnableIT\_FE

## LL\_DMA\_DisableIT\_HT

### Function name

```
__STATIC_INLINE void LL_DMA_DisableIT_HT (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Disable Half transfer interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR HTIE LL\_DMA\_DisableIT\_HT

## LL\_DMA\_DisableIT\_TE

### Function name

```
__STATIC_INLINE void LL_DMA_DisableIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Disable Transfer error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR TEIE LL\_DMA\_DisableIT\_TE

## LL\_DMA\_DisableIT\_TC

### Function name

```
__STATIC_INLINE void LL_DMA_DisableIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Disable Transfer complete interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR TCIE LL\_DMA\_DisableIT\_TC

## LL\_DMA\_DisableIT\_DME

### Function name

```
__STATIC_INLINE void LL_DMA_DisableIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Disable Direct mode error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR DMEIE LL\_DMA\_DisableIT\_DME

## LL\_DMA\_DisableIT\_FE

### Function name

```
__STATIC_INLINE void LL_DMA_DisableIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Disable FIFO error interrupt.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- FCR FEIE LL\_DMA\_DisableIT\_FE

## LL\_DMA\_IsEnabledIT\_HT

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_HT (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Check if Half transfer interrupt is enabled.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- CR HTIE LL\_DMA\_IsEnabledIT\_HT

## LL\_DMA\_IsEnabledIT\_TE

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Check if Transfer error interrupt is enabled.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- CR TEIE LL\_DMA\_IsEnabledIT\_TE

## LL\_DMA\_IsEnabledIT\_TC

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Check if Transfer complete interrupt is enabled.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- CR TCIE LL\_DMA\_IsEnabledIT\_TC

## LL\_DMA\_IsEnabledIT\_DME

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Check if Direct mode error interrupt is enabled.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- CR DMEIE LL\_DMA\_IsEnabledIT\_DME

## LL\_DMA\_IsEnabledIT\_FE

### Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

Check if FIFO error interrupt is enabled.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- FCR FEIE LL\_DMA\_IsEnabledIT\_FE

## LL\_DMA\_Init

### Function name

```
uint32_t LL_DMA_Init (DMA_TypeDef * DMAx, uint32_t Stream, LL_DMA_InitTypeDef * DMA_InitStruct)
```

### Function description

Initialize the DMA registers according to the specified parameters in DMA\_InitStruct.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
- **DMA\_InitStruct:** pointer to a LL\_DMA\_InitTypeDef structure.

### Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: DMA registers are initialized
  - ERROR: Not applicable

### Notes

- To convert DMAx\_Streamy Instance to DMAx Instance and Streamy, use helper macros :  
\_\_LL\_DMA\_GET\_INSTANCE \_\_LL\_DMA\_GET\_STREAM

## LL\_DMA\_DeInit

### Function name

```
uint32_t LL_DMA_DeInit (DMA_TypeDef * DMAx, uint32_t Stream)
```

### Function description

De-initialize the DMA registers to their default reset values.

### Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
  - LL\_DMA\_STREAM\_0
  - LL\_DMA\_STREAM\_1
  - LL\_DMA\_STREAM\_2
  - LL\_DMA\_STREAM\_3
  - LL\_DMA\_STREAM\_4
  - LL\_DMA\_STREAM\_5
  - LL\_DMA\_STREAM\_6
  - LL\_DMA\_STREAM\_7
  - LL\_DMA\_STREAM\_ALL

## Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: DMA registers are de-initialized
  - ERROR: DMA registers are not de-initialized

## LL\_DMA\_StructInit

## Function name

**void LL\_DMA\_StructInit (LL\_DMA\_InitTypeDef \* DMA\_InitStruct)**

## Function description

Set each LL\_DMA\_InitTypeDef field to default value.

## Parameters

- **DMA\_InitStruct:** Pointer to a LL\_DMA\_InitTypeDef structure.

## Return values

- **None:**

## 79.3 DMA Firmware driver defines

The following section lists the various define and macros of the module.

### 79.3.1 DMA DMA CHANNEL

LL\_DMA\_CHANNEL\_0

LL\_DMA\_CHANNEL\_1

LL\_DMA\_CHANNEL\_2

LL\_DMA\_CHANNEL\_3

LL\_DMA\_CHANNEL\_4

LL\_DMA\_CHANNEL\_5

LL\_DMA\_CHANNEL\_6

LL\_DMA\_CHANNEL\_7

### CURRENTTARGETMEM

LL\_DMA\_CURRENTTARGETMEM0

Set CurrentTarget Memory to Memory 0

LL\_DMA\_CURRENTTARGETMEM1

Set CurrentTarget Memory to Memory 1

### DIRECTION

LL\_DMA\_DIRECTION\_PERIPH\_TO\_MEMORY

Peripheral to memory direction



**LL\_DMA\_DIRECTION\_MEMORY\_TO\_PERIPH**

Memory to peripheral direction

**LL\_DMA\_DIRECTION\_MEMORY\_TO\_MEMORY**

Memory to memory direction

**DOUBLEBUFFER MODE****LL\_DMA\_DOUBLEBUFFER\_MODE\_DISABLE**

Disable double buffering mode

**LL\_DMA\_DOUBLEBUFFER\_MODE\_ENABLE**

Enable double buffering mode

**FIFOSTATUS 0****LL\_DMA\_FIFOSTATUS\_0\_25** $0 < \text{fifo\_level} < 1/4$ **LL\_DMA\_FIFOSTATUS\_25\_50** $1/4 < \text{fifo\_level} < 1/2$ **LL\_DMA\_FIFOSTATUS\_50\_75** $1/2 < \text{fifo\_level} < 3/4$ **LL\_DMA\_FIFOSTATUS\_75\_100** $3/4 < \text{fifo\_level} < \text{full}$ **LL\_DMA\_FIFOSTATUS\_EMPTY**

FIFO is empty

**LL\_DMA\_FIFOSTATUS\_FULL**

FIFO is full

**FIFOTHRESHOLD****LL\_DMA\_FIFOTHRESHOLD\_1\_4**

FIFO threshold 1 quart full configuration

**LL\_DMA\_FIFOTHRESHOLD\_1\_2**

FIFO threshold half full configuration

**LL\_DMA\_FIFOTHRESHOLD\_3\_4**

FIFO threshold 3 quarts full configuration

**LL\_DMA\_FIFOTHRESHOLD\_FULL**

FIFO threshold full configuration

**MBURST****LL\_DMA\_MBURST\_SINGLE**

Memory burst single transfer configuration

**LL\_DMA\_MBURST\_INC4**

Memory burst of 4 beats transfer configuration

**LL\_DMA\_MBURST\_INC8**

Memory burst of 8 beats transfer configuration

**LL\_DMA\_MBURST\_INC16**

Memory burst of 16 beats transfer configuration

**MDATAALIGN****LL\_DMA\_MDATAALIGN\_BYTE**

Memory data alignment : Byte

**LL\_DMA\_MDATAALIGN\_HALFWORD**

Memory data alignment : HalfWord

**LL\_DMA\_MDATAALIGN\_WORD**

Memory data alignment : Word

**MEMORY****LL\_DMA\_MEMORY\_NOINCREMENT**

Memory increment mode Disable

**LL\_DMA\_MEMORY\_INCREMENT**

Memory increment mode Enable

**MODE****LL\_DMA\_MODE\_NORMAL**

Normal Mode

**LL\_DMA\_MODE\_CIRCULAR**

Circular Mode

**LL\_DMA\_MODE\_PFCTRL**

Peripheral flow control mode

**OFFSETSIZE****LL\_DMA\_OFFSETSIZE\_PSIZE**

Peripheral increment offset size is linked to the PSIZE

**LL\_DMA\_OFFSETSIZE\_FIXEDTO4**

Peripheral increment offset size is fixed to 4 (32-bit alignment)

**PBURST****LL\_DMA\_PBURST\_SINGLE**

Peripheral burst single transfer configuration

**LL\_DMA\_PBURST\_INC4**

Peripheral burst of 4 beats transfer configuration

**LL\_DMA\_PBURST\_INC8**

Peripheral burst of 8 beats transfer configuration

**LL\_DMA\_PBURST\_INC16**

Peripheral burst of 16 beats transfer configuration

**PDATAALIGN**

**LL\_DMA\_PDATAALIGN\_BYTE**

Peripheral data alignment : Byte

**LL\_DMA\_PDATAALIGN\_HALFWORD**

Peripheral data alignment : HalfWord

**LL\_DMA\_PDATAALIGN\_WORD**

Peripheral data alignment : Word

**PERIPH****LL\_DMA\_PERIPH\_NOINCREMENT**

Peripheral increment mode Disable

**LL\_DMA\_PERIPH\_INCREMENT**

Peripheral increment mode Enable

**PRIORITY****LL\_DMA\_PRIORITY\_LOW**

Priority level : Low

**LL\_DMA\_PRIORITY\_MEDIUM**

Priority level : Medium

**LL\_DMA\_PRIORITY\_HIGH**

Priority level : High

**LL\_DMA\_PRIORITY\_VERYHIGH**

Priority level : Very\_High

**STREAM****LL\_DMA\_STREAM\_0****LL\_DMA\_STREAM\_1****LL\_DMA\_STREAM\_2****LL\_DMA\_STREAM\_3****LL\_DMA\_STREAM\_4****LL\_DMA\_STREAM\_5****LL\_DMA\_STREAM\_6****LL\_DMA\_STREAM\_7****LL\_DMA\_STREAM\_ALL****Convert DMAxStreamy**

## \_\_LL\_DMA\_GET\_INSTANCE

### Description:

- Convert DMAx\_Streamy into DMAx.

### Parameters:

- \_\_STREAM\_INSTANCE\_\_: DMAx\_Streamy

### Return value:

- DMAx

## \_\_LL\_DMA\_GET\_STREAM

### Description:

- Convert DMAx\_Streamy into LL\_DMA\_STREAM\_y.

### Parameters:

- \_\_STREAM\_INSTANCE\_\_: DMAx\_Streamy

### Return value:

- LL\_DMA\_CHANNEL\_y

## \_\_LL\_DMA\_GET\_STREAM\_INSTANCE

### Description:

- Convert DMA Instance DMAx and LL\_DMA\_STREAM\_y into DMAx\_Streamy.

### Parameters:

- \_\_DMA\_INSTANCE\_\_: DMAx
- \_\_STREAM\_\_: LL\_DMA\_STREAM\_y

### Return value:

- DMAx\_Streamy

## Common Write and read registers macros

## LL\_DMA\_WriteReg

### Description:

- Write a value in DMA register.

### Parameters:

- \_\_INSTANCE\_\_: DMA Instance
- \_\_REG\_\_: Register to be written
- \_\_VALUE\_\_: Value to be written in the register

### Return value:

- None

## LL\_DMA\_ReadReg

### Description:

- Read a value in DMA register.

### Parameters:

- \_\_INSTANCE\_\_: DMA Instance
- \_\_REG\_\_: Register to be read

### Return value:

- Register: value

## DMA\_LL\_FIFOMODE

## LL\_DMA\_FIFOMODE\_DISABLE

FIFO mode disable (direct mode is enabled)



LL\_DMA\_FIFOMODE\_ENABLE

FIFO mode enable

## 80 LL EXTI Generic Driver

### 80.1 EXTI Firmware driver registers structures

#### 80.1.1 LL\_EXTI\_InitTypeDef

*LL\_EXTI\_InitTypeDef* is defined in the `stm32f4xx_ll_exti.h`

Data Fields

- *uint32\_t Line\_0\_31*
- *FunctionalState LineCommand*
- *uint8\_t Mode*
- *uint8\_t Trigger*

Field Documentation

- *uint32\_t LL\_EXTI\_InitTypeDef::Line\_0\_31*  
Specifies the EXTI lines to be enabled or disabled for Lines in range 0 to 31 This parameter can be any combination of *EXTI\_LL\_EC\_LINE*
- *FunctionalState LL\_EXTI\_InitTypeDef::LineCommand*  
Specifies the new state of the selected EXTI lines. This parameter can be set either to ENABLE or DISABLE
- *uint8\_t LL\_EXTI\_InitTypeDef::Mode*  
Specifies the mode for the EXTI lines. This parameter can be a value of *EXTI\_LL\_EC\_MODE*.
- *uint8\_t LL\_EXTI\_InitTypeDef::Trigger*  
Specifies the trigger signal active edge for the EXTI lines. This parameter can be a value of *EXTI\_LL\_EC\_TRIGGER*.

### 80.2 EXTI Firmware driver API description

The following section lists the various functions of the EXTI library.

#### 80.2.1 Detailed description of functions

*LL\_EXTI\_EnableIT\_0\_31*

Function name

`__STATIC_INLINE void LL_EXTI_EnableIT_0_31 (uint32_t ExtiLine)`

Function description

Enable ExtiLine Interrupt request for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **None:**

## Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- IMR IMx LL\_EXTI\_EnableIT\_0\_31

## LL\_EXTI\_DisableIT\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_DisableIT\_0\_31 (uint32\_t ExtiLine)**

## Function description

Disable ExtiLine Interrupt request for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **None:**

## Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- IMR IMx LL\_EXTI\_DisableIT\_0\_31

**LL\_EXTI\_IsEnabledIT\_0\_31**

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_EXTI\_IsEnabledIT\_0\_31 (uint32\_t ExtiLine)**

## Function description

Indicate if ExtiLine Interrupt request is enabled for Lines in range 0 to 31.



## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **State:** of bit (1 or 0).

## Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- IMR IMx LL\_EXTI\_IsEnabledIT\_0\_31

### LL\_EXTI\_EnableEvent\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_EnableEvent\_0\_31 (uint32\_t ExtiLine)**

## Function description

Enable ExtiLine Event request for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **None:**

## Notes

- (\*) : Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- EMR EMx LL\_EXTI\_EnableEvent\_0\_31

## LL\_EXTI\_DisableEvent\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_DisableEvent\_0\_31 (uint32\_t ExtiLine)**

## Function description

Disable ExtiLine Event request for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **None:**

## Notes

- (\*) : Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- EMR EMx LL\_EXTI\_DisableEvent\_0\_31

### LL\_EXTI\_IsEnabledEvent\_0\_31

## Function name

`__STATIC_INLINE uint32_t LL_EXTI_IsEnabledEvent_0_31 (uint32_t ExtiLine)`

## Function description

Indicate if ExtiLine Event request is enabled for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be one of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_17
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22
  - LL\_EXTI\_LINE\_23(\*)
  - LL\_EXTI\_LINE\_ALL\_0\_31

## Return values

- **State:** of bit (1 or 0).

## Notes

- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- EMR EMx LL\_EXTI\_IsEnabledEvent\_0\_31

### LL\_EXTI\_EnableRisingTrig\_0\_31

## Function name

`__STATIC_INLINE void LL_EXTI_EnableRisingTrig_0_31 (uint32_t ExtiLine)`

## Function description

Enable ExtiLine Rising Edge Trigger for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a rising edge on a configurable interrupt line occurs during a write operation in the EXTI\_RTISR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- RTSR RTx LL\_EXTI\_EnableRisingTrig\_0\_31

### LL\_EXTI\_DisableRisingTrig\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_DisableRisingTrig\_0\_31 (uint32\_t ExtiLine)**

## Function description

Disable ExtiLine Rising Edge Trigger for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a rising edge on a configurable interrupt line occurs during a write operation in the EXTI\_RTISR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- RTSR RTx LL\_EXTI\_DisableRisingTrig\_0\_31

### LL\_EXTI\_IsEnabledRisingTrig\_0\_31

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_EXTI\_IsEnabledRisingTrig\_0\_31 (uint32\_t ExtiLine)**

## Function description

Check if rising edge trigger is enabled for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **State:** of bit (1 or 0).

## Notes

- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- RTSR RTx LL\_EXTI\_IsEnabledRisingTrig\_0\_31

## LL\_EXTI\_EnableFallingTrig\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_EnableFallingTrig\_0\_31 (uint32\_t ExtiLine)**

## Function description

Enable ExtiLine Falling Edge Trigger for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a falling edge on a configurable interrupt line occurs during a write operation in the EXTI\_FTSR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- FTSR FTx LL\_EXTI\_EnableFallingTrig\_0\_31

### LL\_EXTI\_DisableFallingTrig\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_DisableFallingTrig\_0\_31 (uint32\_t ExtiLine)**

## Function description

Disable ExtiLine Falling Edge Trigger for Lines in range 0 to 31.



## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a Falling edge on a configurable interrupt line occurs during a write operation in the EXTI\_FTSR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- FTSR FTx LL\_EXTI\_DisableFallingTrig\_0\_31

### LL\_EXTI\_IsEnabledFallingTrig\_0\_31

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_EXTI\_IsEnabledFallingTrig\_0\_31 (uint32\_t ExtiLine)**

## Function description

Check if falling edge trigger is enabled for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **State:** of bit (1 or 0).

## Notes

- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- FTSR FTx LL\_EXTI\_IsEnabledFallingTrig\_0\_31

## LL\_EXTI\_GenerateSWI\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_GenerateSWI\_0\_31 (uint32\_t ExtiLine)**

## Function description

Generate a software Interrupt Event for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- If the interrupt is enabled on this line in the EXTI\_IMR, writing a 1 to this bit when it is at '0' sets the corresponding pending bit in EXTI\_PR resulting in an interrupt request generation. This bit is cleared by clearing the corresponding bit in the EXTI\_PR register (by writing a 1 into the bit)
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- SWIER SWIx LL\_EXTI\_GenerateSWI\_0\_31

### LL\_EXTI\_IsActiveFlag\_0\_31

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_EXTI\_IsActiveFlag\_0\_31 (uint32\_t ExtiLine)**

## Function description

Check if the ExtLine Flag is set or not for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **State:** of bit (1 or 0).

## Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- PR PIFx LL\_EXTI\_IsActiveFlag\_0\_31

### LL\_EXTI\_ReadFlag\_0\_31

## Function name

`__STATIC_INLINE uint32_t LL_EXTI_ReadFlag_0_31 (uint32_t ExtiLine)`

## Function description

Read ExtLine Combination Flag for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **@note:** This bit is set when the selected edge event arrives on the interrupt

## Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- PR PIFx LL\_EXTI\_ReadFlag\_0\_31

## LL\_EXTI\_ClearFlag\_0\_31

## Function name

**\_\_STATIC\_INLINE void LL\_EXTI\_ClearFlag\_0\_31 (uint32\_t ExtiLine)**

## Function description

Clear ExtLine Flags for Lines in range 0 to 31.

## Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
  - LL\_EXTI\_LINE\_0
  - LL\_EXTI\_LINE\_1
  - LL\_EXTI\_LINE\_2
  - LL\_EXTI\_LINE\_3
  - LL\_EXTI\_LINE\_4
  - LL\_EXTI\_LINE\_5
  - LL\_EXTI\_LINE\_6
  - LL\_EXTI\_LINE\_7
  - LL\_EXTI\_LINE\_8
  - LL\_EXTI\_LINE\_9
  - LL\_EXTI\_LINE\_10
  - LL\_EXTI\_LINE\_11
  - LL\_EXTI\_LINE\_12
  - LL\_EXTI\_LINE\_13
  - LL\_EXTI\_LINE\_14
  - LL\_EXTI\_LINE\_15
  - LL\_EXTI\_LINE\_16
  - LL\_EXTI\_LINE\_18
  - LL\_EXTI\_LINE\_19(\*)
  - LL\_EXTI\_LINE\_20(\*)
  - LL\_EXTI\_LINE\_21
  - LL\_EXTI\_LINE\_22

## Return values

- **None:**

## Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- (\*): Available in some devices
- Please check each device line mapping for EXTI Line availability

## Reference Manual to LL API cross reference:

- PR PIFx LL\_EXTI\_ClearFlag\_0\_31

## LL\_EXTI\_Init

## Function name

uint32\_t LL\_EXTI\_Init (LL\_EXTI\_InitTypeDef \* EXTI\_InitStruct)

## Function description

Initialize the EXTI registers according to the specified parameters in EXTI\_InitStruct.

## Parameters

- **EXTI\_InitStruct:** pointer to a LL\_EXTI\_InitTypeDef structure.

## Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: EXTI registers are initialized
  - ERROR: not applicable

## LL\_EXTI\_DeInit

### Function name

uint32\_t LL\_EXTI\_DeInit (void )

### Function description

De-initialize the EXTI registers to their default reset values.

### Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: EXTI registers are de-initialized
  - ERROR: not applicable

## LL\_EXTI\_StructInit

### Function name

void LL\_EXTI\_StructInit (LL\_EXTI\_InitTypeDef \* EXTI\_InitStruct)

### Function description

Set each LL\_EXTI\_InitTypeDef field to default value.

### Parameters

- **EXTI\_InitStruct:** Pointer to a LL\_EXTI\_InitTypeDef structure.

### Return values

- **None:**

## 80.3 EXTI Firmware driver defines

The following section lists the various define and macros of the module.

### 80.3.1 EXTI

EXTI  
**LINE**

#### LL\_EXTI\_LINE\_0

Extended line 0

#### LL\_EXTI\_LINE\_1

Extended line 1

#### LL\_EXTI\_LINE\_2

Extended line 2

#### LL\_EXTI\_LINE\_3

Extended line 3

#### LL\_EXTI\_LINE\_4

Extended line 4

#### LL\_EXTI\_LINE\_5

Extended line 5

#### LL\_EXTI\_LINE\_6

Extended line 6

**LL\_EXTI\_LINE\_7**

Extended line 7

**LL\_EXTI\_LINE\_8**

Extended line 8

**LL\_EXTI\_LINE\_9**

Extended line 9

**LL\_EXTI\_LINE\_10**

Extended line 10

**LL\_EXTI\_LINE\_11**

Extended line 11

**LL\_EXTI\_LINE\_12**

Extended line 12

**LL\_EXTI\_LINE\_13**

Extended line 13

**LL\_EXTI\_LINE\_14**

Extended line 14

**LL\_EXTI\_LINE\_15**

Extended line 15

**LL\_EXTI\_LINE\_16**

Extended line 16

**LL\_EXTI\_LINE\_17**

Extended line 17

**LL\_EXTI\_LINE\_18**

Extended line 18

**LL\_EXTI\_LINE\_19**

Extended line 19

**LL\_EXTI\_LINE\_20**

Extended line 20

**LL\_EXTI\_LINE\_21**

Extended line 21

**LL\_EXTI\_LINE\_22**

Extended line 22

**LL\_EXTI\_LINE\_ALL\_0\_31**

All Extended line not reserved

**LL\_EXTI\_LINE\_ALL**

All Extended line

**LL\_EXTI\_LINE\_NONE**

None Extended line

**Mode**



**LL\_EXTI\_MODE\_IT**

Interrupt Mode

**LL\_EXTI\_MODE\_EVENT**

Event Mode

**LL\_EXTI\_MODE\_IT\_EVENT**

Interrupt &amp; Event Mode

**Edge Trigger****LL\_EXTI\_TRIGGER\_NONE**

No Trigger Mode

**LL\_EXTI\_TRIGGER\_RISING**

Trigger Rising Mode

**LL\_EXTI\_TRIGGER\_FALLING**

Trigger Falling Mode

**LL\_EXTI\_TRIGGER\_RISING\_FALLING**

Trigger Rising &amp; Falling Mode

**Common Write and read registers Macros****LL\_EXTI\_WriteReg****Description:**

- Write a value in EXTI register.

**Parameters:**

- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

**Return value:**

- None

**LL\_EXTI\_ReadReg****Description:**

- Read a value in EXTI register.

**Parameters:**

- `__REG__`: Register to be read

**Return value:**

- Register: value

## 82 LL GPIO Generic Driver

### 82.1 GPIO Firmware driver registers structures

#### 82.1.1 LL\_GPIO\_InitTypeDef

*LL\_GPIO\_InitTypeDef* is defined in the `stm32f4xx_ll_gpio.h`

##### Data Fields

- *uint32\_t* Pin
- *uint32\_t* Mode
- *uint32\_t* Speed
- *uint32\_t* OutputType
- *uint32\_t* Pull
- *uint32\_t* Alternate

##### Field Documentation

- *uint32\_t* *LL\_GPIO\_InitTypeDef::Pin*  
Specifies the GPIO pins to be configured. This parameter can be any value of [GPIO\\_LL\\_EC\\_PIN](#)
- *uint32\_t* *LL\_GPIO\_InitTypeDef::Mode*  
Specifies the operating mode for the selected pins. This parameter can be a value of [GPIO\\_LL\\_EC\\_MODE](#).GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinMode()`.
- *uint32\_t* *LL\_GPIO\_InitTypeDef::Speed*  
Specifies the speed for the selected pins. This parameter can be a value of [GPIO\\_LL\\_EC\\_SPEED](#).GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinSpeed()`.
- *uint32\_t* *LL\_GPIO\_InitTypeDef::OutputType*  
Specifies the operating output type for the selected pins. This parameter can be a value of [GPIO\\_LL\\_EC\\_OUTPUT](#).GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinOutputType()`.
- *uint32\_t* *LL\_GPIO\_InitTypeDef::Pull*  
Specifies the operating Pull-up/Pull down for the selected pins. This parameter can be a value of [GPIO\\_LL\\_EC\\_PULL](#).GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinPull()`.
- *uint32\_t* *LL\_GPIO\_InitTypeDef::Alternate*  
Specifies the Peripheral to be connected to the selected pins. This parameter can be a value of [GPIO\\_LL\\_EC\\_AF](#).GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetAFPin_0_7()` and `LL_GPIO_SetAFPin_8_15()`.

### 82.2 GPIO Firmware driver API description

The following section lists the various functions of the GPIO library.

#### 82.2.1 Detailed description of functions

##### LL\_GPIO\_SetPinMode

##### Function name

```
__STATIC_INLINE void LL_GPIO_SetPinMode (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Mode)
```

##### Function description

Configure gpio mode for a dedicated pin on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
- **Mode:** This parameter can be one of the following values:
  - LL\_GPIO\_MODE\_INPUT
  - LL\_GPIO\_MODE\_OUTPUT
  - LL\_GPIO\_MODE\_ALTERNATE
  - LL\_GPIO\_MODE\_ANALOG

## Return values

- **None:**

## Notes

- I/O mode can be Input mode, General purpose output, Alternate function mode or Analog.
- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- MODER MODEy LL\_GPIO\_SetPinMode

## LL\_GPIO\_GetPinMode

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_GPIO\_GetPinMode (GPIO\_TypeDef \* GPIOx, uint32\_t Pin)**

## Function description

Return gpio mode for a dedicated pin on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_MODE\_INPUT
  - LL\_GPIO\_MODE\_OUTPUT
  - LL\_GPIO\_MODE\_ALTERNATE
  - LL\_GPIO\_MODE\_ANALOG

## Notes

- I/O mode can be Input mode, General purpose output, Alternate function mode or Analog.
- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- MODER MODEy LL\_GPIO\_GetPinMode

### LL\_GPIO\_SetPinOutputType

## Function name

```
__STATIC_INLINE void LL_GPIO_SetPinOutputType (GPIO_TypeDef * GPIOx, uint32_t PinMask, uint32_t OutputType)
```

## Function description

Configure gpio output type for several pins on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL
- **OutputType:** This parameter can be one of the following values:
  - LL\_GPIO\_OUTPUT\_PUSHPULL
  - LL\_GPIO\_OUTPUT\_OPENDRAIN

## Return values

- **None:**

## Notes

- Output type as to be set when gpio pin is in output or alternate modes. Possible type are Push-pull or Open-drain.

## Reference Manual to LL API cross reference:

- OTYPER OTy LL\_GPIO\_SetPinOutputType

### LL\_GPIO\_GetPinOutputType

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_GPIO\_GetPinOutputType (GPIO\_TypeDef \* GPIOx, uint32\_t Pin)**

## Function description

Return gpio output type for several pins on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_OUTPUT\_PUSHPULL
  - LL\_GPIO\_OUTPUT\_OPENDRAIN

## Notes

- Output type as to be set when gpio pin is in output or alternate modes. Possible type are Push-pull or Open-drain.
- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- OTYPER OTy LL\_GPIO\_GetPinOutputType

### LL\_GPIO\_SetPinSpeed

## Function name

**\_\_STATIC\_INLINE void LL\_GPIO\_SetPinSpeed (GPIO\_TypeDef \* GPIOx, uint32\_t Pin, uint32\_t Speed)**

## Function description

Configure gpio speed for a dedicated pin on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
- **Speed:** This parameter can be one of the following values:
  - LL\_GPIO\_SPEED\_FREQ\_LOW
  - LL\_GPIO\_SPEED\_FREQ\_MEDIUM
  - LL\_GPIO\_SPEED\_FREQ\_HIGH
  - LL\_GPIO\_SPEED\_FREQ\_VERY\_HIGH

## Return values

- **None:**

## Notes

- I/O speed can be Low, Medium, Fast or High speed.
- Warning: only one pin can be passed as parameter.
- Refer to datasheet for frequency specifications and the power supply and load conditions for each speed.

## Reference Manual to LL API cross reference:

- OSPEEDR OSPEEDy LL\_GPIO\_SetPinSpeed

## LL\_GPIO\_GetPinSpeed

## Function name

```
__STATIC_INLINE uint32_t LL_GPIO_GetPinSpeed(GPIO_TypeDef * GPIOx, uint32_t Pin)
```

## Function description

Return gpio speed for a dedicated pin on dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_SPEED\_FREQ\_LOW
  - LL\_GPIO\_SPEED\_FREQ\_MEDIUM
  - LL\_GPIO\_SPEED\_FREQ\_HIGH
  - LL\_GPIO\_SPEED\_FREQ\_VERY\_HIGH

## Notes

- I/O speed can be Low, Medium, Fast or High speed.
- Warning: only one pin can be passed as parameter.
- Refer to datasheet for frequency specifications and the power supply and load conditions for each speed.

## Reference Manual to LL API cross reference:

- OSPEEDR OSPEEDy LL\_GPIO\_GetPinSpeed

## LL\_GPIO\_SetPinPull

### Function name

**\_\_STATIC\_INLINE void LL\_GPIO\_SetPinPull (GPIO\_TypeDef \* GPIOx, uint32\_t Pin, uint32\_t Pull)**

### Function description

Configure gpio pull-up or pull-down for a dedicated pin on a dedicated port.



## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
- **Pull:** This parameter can be one of the following values:
  - LL\_GPIO\_PULL\_NO
  - LL\_GPIO\_PULL\_UP
  - LL\_GPIO\_PULL\_DOWN

## Return values

- **None:**

## Notes

- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- PUPDR PUPDy LL\_GPIO\_SetPinPull

## LL\_GPIO\_GetPinPull

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_GPIO\_GetPinPull (GPIO\_TypeDef \* GPIOx, uint32\_t Pin)**

## Function description

Return gpio pull-up or pull-down for a dedicated pin on a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_PULL\_NO
  - LL\_GPIO\_PULL\_UP
  - LL\_GPIO\_PULL\_DOWN

## Notes

- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- PUPDR PUPDy LL\_GPIO\_GetPinPull

### LL\_GPIO\_SetAFPin\_0\_7

## Function name

**\_\_STATIC\_INLINE void LL\_GPIO\_SetAFPin\_0\_7 (GPIO\_TypeDef \* GPIOx, uint32\_t Pin, uint32\_t Alternate)**

## Function description

Configure gpio alternate function of a dedicated pin from 0 to 7 for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
- **Alternate:** This parameter can be one of the following values:
  - LL\_GPIO\_AF\_0
  - LL\_GPIO\_AF\_1
  - LL\_GPIO\_AF\_2
  - LL\_GPIO\_AF\_3
  - LL\_GPIO\_AF\_4
  - LL\_GPIO\_AF\_5
  - LL\_GPIO\_AF\_6
  - LL\_GPIO\_AF\_7
  - LL\_GPIO\_AF\_8
  - LL\_GPIO\_AF\_9
  - LL\_GPIO\_AF\_10
  - LL\_GPIO\_AF\_11
  - LL\_GPIO\_AF\_12
  - LL\_GPIO\_AF\_13
  - LL\_GPIO\_AF\_14
  - LL\_GPIO\_AF\_15

## Return values

- **None:**

## Notes

- Possible values are from AF0 to AF15 depending on target.
- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- AFRL AFSELY LL\_GPIO\_SetAFPin\_0\_7

### LL\_GPIO\_GetAFPin\_0\_7

## Function name

`__STATIC_INLINE uint32_t LL_GPIO_GetAFPin_0_7 (GPIO_TypeDef * GPIOx, uint32_t Pin)`

## Function description

Return gpio alternate function of a dedicated pin from 0 to 7 for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_AF\_0
  - LL\_GPIO\_AF\_1
  - LL\_GPIO\_AF\_2
  - LL\_GPIO\_AF\_3
  - LL\_GPIO\_AF\_4
  - LL\_GPIO\_AF\_5
  - LL\_GPIO\_AF\_6
  - LL\_GPIO\_AF\_7
  - LL\_GPIO\_AF\_8
  - LL\_GPIO\_AF\_9
  - LL\_GPIO\_AF\_10
  - LL\_GPIO\_AF\_11
  - LL\_GPIO\_AF\_12
  - LL\_GPIO\_AF\_13
  - LL\_GPIO\_AF\_14
  - LL\_GPIO\_AF\_15

## Reference Manual to LL API cross reference:

- AFRL AFSELY LL\_GPIO\_GetAFPin\_0\_7

### LL\_GPIO\_SetAFPin\_8\_15

## Function name

```
__STATIC_INLINE void LL_GPIO_SetAFPin_8_15 (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Alternate)
```

## Function description

Configure gpio alternate function of a dedicated pin from 8 to 15 for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
- **Alternate:** This parameter can be one of the following values:
  - LL\_GPIO\_AF\_0
  - LL\_GPIO\_AF\_1
  - LL\_GPIO\_AF\_2
  - LL\_GPIO\_AF\_3
  - LL\_GPIO\_AF\_4
  - LL\_GPIO\_AF\_5
  - LL\_GPIO\_AF\_6
  - LL\_GPIO\_AF\_7
  - LL\_GPIO\_AF\_8
  - LL\_GPIO\_AF\_9
  - LL\_GPIO\_AF\_10
  - LL\_GPIO\_AF\_11
  - LL\_GPIO\_AF\_12
  - LL\_GPIO\_AF\_13
  - LL\_GPIO\_AF\_14
  - LL\_GPIO\_AF\_15

## Return values

- **None:**

## Notes

- Possible values are from AF0 to AF15 depending on target.
- Warning: only one pin can be passed as parameter.

## Reference Manual to LL API cross reference:

- AFRH AFSELY LL\_GPIO\_SetAFPin\_8\_15

### LL\_GPIO\_GetAFPin\_8\_15

## Function name

`__STATIC_INLINE uint32_t LL_GPIO_GetAFPin_8_15 (GPIO_TypeDef * GPIOx, uint32_t Pin)`

## Function description

Return gpio alternate function of a dedicated pin from 8 to 15 for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15

## Return values

- **Returned:** value can be one of the following values:
  - LL\_GPIO\_AF\_0
  - LL\_GPIO\_AF\_1
  - LL\_GPIO\_AF\_2
  - LL\_GPIO\_AF\_3
  - LL\_GPIO\_AF\_4
  - LL\_GPIO\_AF\_5
  - LL\_GPIO\_AF\_6
  - LL\_GPIO\_AF\_7
  - LL\_GPIO\_AF\_8
  - LL\_GPIO\_AF\_9
  - LL\_GPIO\_AF\_10
  - LL\_GPIO\_AF\_11
  - LL\_GPIO\_AF\_12
  - LL\_GPIO\_AF\_13
  - LL\_GPIO\_AF\_14
  - LL\_GPIO\_AF\_15

## Notes

- Possible values are from AF0 to AF15 depending on target.

## Reference Manual to LL API cross reference:

- AFRH AFSELY LL\_GPIO\_GetAFPin\_8\_15

### LL\_GPIO\_LockPin

## Function name

```
__STATIC_INLINE void LL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

## Function description

Lock configuration of several pins for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

## Return values

- **None:**

## Notes

- When the lock sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next reset.
- Each lock bit freezes a specific configuration register (control and alternate function registers).

## Reference Manual to LL API cross reference:

- LCKR LCKK LL\_GPIO\_LockPin

## LL\_GPIO\_IsPinLocked

## Function name

```
__STATIC_INLINE uint32_t LL_GPIO_IsPinLocked (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

## Function description

Return 1 if all pins passed as parameter, of a dedicated port, are locked.

## Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- LCKR LCKy LL\_GPIO\_IsPinLocked

### LL\_GPIO\_IsAnyPinLocked

## Function name

```
__STATIC_INLINE uint32_t LL_GPIO_IsAnyPinLocked (GPIO_TypeDef * GPIOx)
```

## Function description

Return 1 if one of the pin of a dedicated port is locked.

## Parameters

- **GPIOx:** GPIO Port

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- LCKR LCKK LL\_GPIO\_IsAnyPinLocked

### LL\_GPIO\_ReadInputPort

## Function name

```
__STATIC_INLINE uint32_t LL_GPIO_ReadInputPort (GPIO_TypeDef * GPIOx)
```

## Function description

Return full input data register value for a dedicated port.

## Parameters

- **GPIOx:** GPIO Port



#### Return values

- **Input:** data register value of port

#### Reference Manual to LL API cross reference:

- IDR IDy LL\_GPIO\_ReadInputPort

#### LL\_GPIO\_IsInputPinSet

#### Function name

```
__STATIC_INLINE uint32_t LL_GPIO_IsInputPinSet (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

#### Function description

Return if input data level for several pins of dedicated port is high or low.

#### Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- IDR IDy LL\_GPIO\_IsInputPinSet

#### LL\_GPIO\_WriteOutputPort

#### Function name

```
__STATIC_INLINE void LL_GPIO_WriteOutputPort (GPIO_TypeDef * GPIOx, uint32_t PortValue)
```

#### Function description

Write output data register for the port.

#### Parameters

- **GPIOx:** GPIO Port
- **PortValue:** Level value for each pin of the port

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- ODR ODy LL\_GPIO\_WriteOutputPort

#### LL\_GPIO\_ReadOutputPort

#### Function name

```
__STATIC_INLINE uint32_t LL_GPIO_ReadOutputPort (GPIO_TypeDef * GPIOx)
```

#### Function description

Return full output data register value for a dedicated port.

#### Parameters

- **GPIOx:** GPIO Port

#### Return values

- **Output:** data register value of port

#### Reference Manual to LL API cross reference:

- ODR ODy LL\_GPIO\_ReadOutputPort

#### LL\_GPIO\_IsOutputPinSet

#### Function name

```
__STATIC_INLINE uint32_t LL_GPIO_IsOutputPinSet (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

#### Function description

Return if input data level for several pins of dedicated port is high or low.

#### Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- ODR ODy LL\_GPIO\_IsOutputPinSet

## LL\_GPIO\_SetOutputPin

### Function name

```
__STATIC_INLINE void LL_GPIO_SetOutputPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

### Function description

Set several pins to high level on dedicated gpio port.

### Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- BSRR BSy LL\_GPIO\_SetOutputPin

## LL\_GPIO\_ResetOutputPin

### Function name

```
__STATIC_INLINE void LL_GPIO_ResetOutputPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

### Function description

Set several pins to low level on dedicated gpio port.

## Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- BSRR BRy LL\_GPIO\_ResetOutputPin

### LL\_GPIO\_TogglePin

## Function name

**\_\_STATIC\_INLINE void LL\_GPIO\_TogglePin (GPIO\_TypeDef \* GPIOx, uint32\_t PinMask)**

## Function description

Toggle data value for several pin of dedicated port.

## Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
  - LL\_GPIO\_PIN\_0
  - LL\_GPIO\_PIN\_1
  - LL\_GPIO\_PIN\_2
  - LL\_GPIO\_PIN\_3
  - LL\_GPIO\_PIN\_4
  - LL\_GPIO\_PIN\_5
  - LL\_GPIO\_PIN\_6
  - LL\_GPIO\_PIN\_7
  - LL\_GPIO\_PIN\_8
  - LL\_GPIO\_PIN\_9
  - LL\_GPIO\_PIN\_10
  - LL\_GPIO\_PIN\_11
  - LL\_GPIO\_PIN\_12
  - LL\_GPIO\_PIN\_13
  - LL\_GPIO\_PIN\_14
  - LL\_GPIO\_PIN\_15
  - LL\_GPIO\_PIN\_ALL

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- ODR ODy LL\_GPIO\_TogglePin

## LL\_GPIO\_DeInit

## Function name

**ErrorStatus LL\_GPIO\_DeInit (GPIO\_TypeDef \* GPIOx)**

## Function description

De-initialize GPIO registers (Registers restored to their default values).

## Parameters

- **GPIOx:** GPIO Port

## Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: GPIO registers are de-initialized
  - ERROR: Wrong GPIO Port

## LL\_GPIO\_Init

## Function name

**ErrorStatus LL\_GPIO\_Init (GPIO\_TypeDef \* GPIOx, LL\_GPIO\_InitTypeDef \* GPIO\_InitStruct)**

## Function description

Initialize GPIO registers according to the specified parameters in GPIO\_InitStruct.

## Parameters

- **GPIOx:** GPIO Port
- **GPIO\_InitStruct:** pointer to a LL\_GPIO\_InitTypeDef structure that contains the configuration information for the specified GPIO peripheral.

## Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: GPIO registers are initialized according to GPIO\_InitStruct content
  - ERROR: Not applicable

## LL\_GPIO\_StructInit

## Function name

**void LL\_GPIO\_StructInit (LL\_GPIO\_InitTypeDef \* GPIO\_InitStruct)**

## Function description

Set each LL\_GPIO\_InitTypeDef field to default value.

## Parameters

- **GPIO\_InitStruct:** pointer to a LL\_GPIO\_InitTypeDef structure whose fields will be set to default values.

## Return values

- **None:**

## 82.3 GPIO Firmware driver defines

The following section lists the various define and macros of the module.

### 82.3.1 GPIO

GPIO

#### *Alternate Function*

#### LL\_GPIO\_AF\_0

Select alternate function 0

#### LL\_GPIO\_AF\_1

Select alternate function 1

#### LL\_GPIO\_AF\_2

Select alternate function 2

#### LL\_GPIO\_AF\_3

Select alternate function 3

#### LL\_GPIO\_AF\_4

Select alternate function 4

#### LL\_GPIO\_AF\_5

Select alternate function 5

#### LL\_GPIO\_AF\_6

Select alternate function 6

#### LL\_GPIO\_AF\_7

Select alternate function 7

**LL\_GPIO\_AF\_8**

Select alternate function 8

**LL\_GPIO\_AF\_9**

Select alternate function 9

**LL\_GPIO\_AF\_10**

Select alternate function 10

**LL\_GPIO\_AF\_11**

Select alternate function 11

**LL\_GPIO\_AF\_12**

Select alternate function 12

**LL\_GPIO\_AF\_13**

Select alternate function 13

**LL\_GPIO\_AF\_14**

Select alternate function 14

**LL\_GPIO\_AF\_15**

Select alternate function 15

**Mode****LL\_GPIO\_MODE\_INPUT**

Select input mode

**LL\_GPIO\_MODE\_OUTPUT**

Select output mode

**LL\_GPIO\_MODE\_ALTERNATE**

Select alternate function mode

**LL\_GPIO\_MODE\_ANALOG**

Select analog mode

**Output Type****LL\_GPIO\_OUTPUT\_PUSH\_PULL**

Select push-pull as output type

**LL\_GPIO\_OUTPUT\_OPENDRAIN**

Select open-drain as output type

**PIN****LL\_GPIO\_PIN\_0**

Select pin 0

**LL\_GPIO\_PIN\_1**

Select pin 1

**LL\_GPIO\_PIN\_2**

Select pin 2

**LL\_GPIO\_PIN\_3**

Select pin 3

**LL\_GPIO\_PIN\_4**

Select pin 4

**LL\_GPIO\_PIN\_5**

Select pin 5

**LL\_GPIO\_PIN\_6**

Select pin 6

**LL\_GPIO\_PIN\_7**

Select pin 7

**LL\_GPIO\_PIN\_8**

Select pin 8

**LL\_GPIO\_PIN\_9**

Select pin 9

**LL\_GPIO\_PIN\_10**

Select pin 10

**LL\_GPIO\_PIN\_11**

Select pin 11

**LL\_GPIO\_PIN\_12**

Select pin 12

**LL\_GPIO\_PIN\_13**

Select pin 13

**LL\_GPIO\_PIN\_14**

Select pin 14

**LL\_GPIO\_PIN\_15**

Select pin 15

**LL\_GPIO\_PIN\_ALL**

Select all pins

***Pull Up Pull Down*****LL\_GPIO\_PULL\_NO**

Select I/O no pull

**LL\_GPIO\_PULL\_UP**

Select I/O pull up

**LL\_GPIO\_PULL\_DOWN**

Select I/O pull down

***Output Speed*****LL\_GPIO\_SPEED\_FREQ\_LOW**

Select I/O low output speed



**LL\_GPIO\_SPEED\_FREQ\_MEDIUM**

Select I/O medium output speed

**LL\_GPIO\_SPEED\_FREQ\_HIGH**

Select I/O fast output speed

**LL\_GPIO\_SPEED\_FREQ\_VERY\_HIGH**

Select I/O high output speed

***Common Write and read registers Macros*****LL\_GPIO\_WriteReg****Description:**

- Write a value in GPIO register.

**Parameters:**

- `__INSTANCE__`: GPIO Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

**Return value:**

- None

**LL\_GPIO\_ReadReg****Description:**

- Read a value in GPIO register.

**Parameters:**

- `__INSTANCE__`: GPIO Instance
- `__REG__`: Register to be read

**Return value:**

- Register: value

## 90 LL SPI Generic Driver

### 90.1 SPI Firmware driver registers structures

#### 90.1.1 LL\_SPI\_InitTypeDef

**LL\_SPI\_InitTypeDef** is defined in the `stm32f4xx_ll_spi.h`

##### Data Fields

- `uint32_t TransferDirection`
- `uint32_t Mode`
- `uint32_t DataWidth`
- `uint32_t ClockPolarity`
- `uint32_t ClockPhase`
- `uint32_t NSS`
- `uint32_t BaudRate`
- `uint32_t BitOrder`
- `uint32_t CRCCalculation`
- `uint32_t CRCPoly`

##### Field Documentation

- `uint32_t LL_SPI_InitTypeDef::TransferDirection`  
Specifies the SPI unidirectional or bidirectional data mode. This parameter can be a value of [SPI\\_LL\\_EC\\_TRANSFER\\_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetTransferDirection()`.
- `uint32_t LL_SPI_InitTypeDef::Mode`  
Specifies the SPI mode (Master/Slave). This parameter can be a value of [SPI\\_LL\\_EC\\_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetMode()`.
- `uint32_t LL_SPI_InitTypeDef::DataWidth`  
Specifies the SPI data width. This parameter can be a value of [SPI\\_LL\\_EC\\_DATAWIDTH](#). This feature can be modified afterwards using unitary function `LL_SPI_SetDataWidth()`.
- `uint32_t LL_SPI_InitTypeDef::ClockPolarity`  
Specifies the serial clock steady state. This parameter can be a value of [SPI\\_LL\\_EC\\_POLARITY](#). This feature can be modified afterwards using unitary function `LL_SPI_SetClockPolarity()`.
- `uint32_t LL_SPI_InitTypeDef::ClockPhase`  
Specifies the clock active edge for the bit capture. This parameter can be a value of [SPI\\_LL\\_EC\\_PHASE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetClockPhase()`.
- `uint32_t LL_SPI_InitTypeDef::NSS`  
Specifies whether the NSS signal is managed by hardware (NSS pin) or by software using the SSI bit. This parameter can be a value of [SPI\\_LL\\_EC\\_NSS\\_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetNSSMode()`.
- `uint32_t LL_SPI_InitTypeDef::BaudRate`  
Specifies the BaudRate prescaler value which will be used to configure the transmit and receive SCK clock. This parameter can be a value of [SPI\\_LL\\_EC\\_BAUDRATEPRESCALER](#).  
**Note:**
  - The communication clock is derived from the master clock. The slave clock does not need to be set. This feature can be modified afterwards using unitary function `LL_SPI_SetBaudRatePrescaler()`.
- `uint32_t LL_SPI_InitTypeDef::BitOrder`  
Specifies whether data transfers start from MSB or LSB bit. This parameter can be a value of [SPI\\_LL\\_EC\\_BIT\\_ORDER](#). This feature can be modified afterwards using unitary function `LL_SPI_SetTransferBitOrder()`.
- `uint32_t LL_SPI_InitTypeDef::CRCCalculation`  
Specifies if the CRC calculation is enabled or not. This parameter can be a value of [SPI\\_LL\\_EC\\_CRC\\_CALCULATION](#). This feature can be modified afterwards using unitary functions `LL_SPI_EnableCRC()` and `LL_SPI_DisableCRC()`.

- **`uint32_t LL_SPI_InitTypeDef::CRCPoly`**  
Specifies the polynomial used for the CRC calculation. This parameter must be a number between `Min_Data = 0x00` and `Max_Data = 0xFFFF`. This feature can be modified afterwards using unitary function `LL_SPI_SetCRCPolynomial()`.

### 90.1.2

#### LL\_I2S\_InitTypeDef

**`LL_I2S_InitTypeDef`** is defined in the `stm32f4xx_ll_spi.h`

##### Data Fields

- **`uint32_t Mode`**
- **`uint32_t Standard`**
- **`uint32_t DataFormat`**
- **`uint32_t MCLKOutput`**
- **`uint32_t AudioFreq`**
- **`uint32_t ClockPolarity`**

##### Field Documentation

- **`uint32_t LL_I2S_InitTypeDef::Mode`**  
Specifies the I2S operating mode. This parameter can be a value of `I2S_LL_EC_MODE`. This feature can be modified afterwards using unitary function `LL_I2S_SetTransferMode()`.
- **`uint32_t LL_I2S_InitTypeDef::Standard`**  
Specifies the standard used for the I2S communication. This parameter can be a value of `I2S_LL_EC_STANDARD`. This feature can be modified afterwards using unitary function `LL_I2S_SetStandard()`.
- **`uint32_t LL_I2S_InitTypeDef::DataFormat`**  
Specifies the data format for the I2S communication. This parameter can be a value of `I2S_LL_EC_DATA_FORMAT`. This feature can be modified afterwards using unitary function `LL_I2S_SetDataFormat()`.
- **`uint32_t LL_I2S_InitTypeDef::MCLKOutput`**  
Specifies whether the I2S MCLK output is enabled or not. This parameter can be a value of `I2S_LL_EC_MCLK_OUTPUT`. This feature can be modified afterwards using unitary functions `LL_I2S_EnableMasterClock()` or `LL_I2S_DisableMasterClock()`.
- **`uint32_t LL_I2S_InitTypeDef::AudioFreq`**  
Specifies the frequency selected for the I2S communication. This parameter can be a value of `I2S_LL_EC_AUDIO_FREQ`. Audio Frequency can be modified afterwards using Reference manual formulas to calculate Prescaler Linear, Parity and unitary functions `LL_I2S_SetPrescalerLinear()` and `LL_I2S_SetPrescalerParity()` to set it.
- **`uint32_t LL_I2S_InitTypeDef::ClockPolarity`**  
Specifies the idle state of the I2S clock. This parameter can be a value of `I2S_LL_EC_POLARITY`. This feature can be modified afterwards using unitary function `LL_I2S_SetClockPolarity()`.

## 90.2

### SPI Firmware driver API description

The following section lists the various functions of the SPI library.

### 90.2.1

#### Detailed description of functions

##### LL\_SPI\_Enable

##### Function name

```
__STATIC_INLINE void LL_SPI_Enable (SPI_TypeDef * SPIx)
```

##### Function description

Enable SPI peripheral.

##### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR1 SPE LL\_SPI\_Enable

#### LL\_SPI\_Disable

#### Function name

```
__STATIC_INLINE void LL_SPI_Disable (SPI_TypeDef * SPIx)
```

#### Function description

Disable SPI peripheral.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Notes

- When disabling the SPI, follow the procedure described in the Reference Manual.

#### Reference Manual to LL API cross reference:

- CR1 SPE LL\_SPI\_Disable

#### LL\_SPI\_IsEnabled

#### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabled (SPI_TypeDef * SPIx)
```

#### Function description

Check if SPI peripheral is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR1 SPE LL\_SPI\_IsEnabled

#### LL\_SPI\_SetMode

#### Function name

```
__STATIC_INLINE void LL_SPI_SetMode (SPI_TypeDef * SPIx, uint32_t Mode)
```

#### Function description

Set SPI operation mode to Master or Slave.

#### Parameters

- **SPIx:** SPI Instance
- **Mode:** This parameter can be one of the following values:
  - LL\_SPI\_MODE\_MASTER
  - LL\_SPI\_MODE\_SLAVE

## Return values

- **None:**

## Notes

- This bit should not be changed when communication is ongoing.

## Reference Manual to LL API cross reference:

- CR1 MSTR LL\_SPI\_SetMode
- CR1 SSI LL\_SPI\_SetMode

## LL\_SPI\_GetMode

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetMode (SPI_TypeDef * SPIx)
```

## Function description

Get SPI operation mode (Master or Slave)

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_MODE\_MASTER
  - LL\_SPI\_MODE\_SLAVE

## Reference Manual to LL API cross reference:

- CR1 MSTR LL\_SPI\_GetMode
- CR1 SSI LL\_SPI\_GetMode

## LL\_SPI\_SetStandard

## Function name

```
__STATIC_INLINE void LL_SPI_SetStandard (SPI_TypeDef * SPIx, uint32_t Standard)
```

## Function description

Set serial protocol used.

## Parameters

- **SPIx:** SPI Instance
- **Standard:** This parameter can be one of the following values:
  - LL\_SPI\_PROTOCOL\_MOTOROLA
  - LL\_SPI\_PROTOCOL\_TI

## Return values

- **None:**

## Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

## Reference Manual to LL API cross reference:

- CR2 FRF LL\_SPI\_SetStandard

## LL\_SPI\_GetStandard

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetStandard (SPI_TypeDef * SPIx)
```

## Function description

Get serial protocol used.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_PROTOCOL\_MOTOROLA
  - LL\_SPI\_PROTOCOL\_TI

## Reference Manual to LL API cross reference:

- CR2 FRF LL\_SPI\_GetStandard

## LL\_SPI\_SetClockPhase

## Function name

```
__STATIC_INLINE void LL_SPI_SetClockPhase (SPI_TypeDef * SPIx, uint32_t ClockPhase)
```

## Function description

Set clock phase.

## Parameters

- **SPIx:** SPI Instance
- **ClockPhase:** This parameter can be one of the following values:
  - LL\_SPI\_PHASE\_1EDGE
  - LL\_SPI\_PHASE\_2EDGE

## Return values

- **None:**

## Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

## Reference Manual to LL API cross reference:

- CR1 CPHA LL\_SPI\_SetClockPhase

## LL\_SPI\_GetClockPhase

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetClockPhase (SPI_TypeDef * SPIx)
```

## Function description

Get clock phase.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_PHASE\_1EDGE
  - LL\_SPI\_PHASE\_2EDGE

## Reference Manual to LL API cross reference:

- CR1 CPHA LL\_SPI\_GetClockPhase

## LL\_SPI\_SetClockPolarity

### Function name

```
__STATIC_INLINE void LL_SPI_SetClockPolarity (SPI_TypeDef * SPIx, uint32_t ClockPolarity)
```

### Function description

Set clock polarity.

### Parameters

- **SPIx:** SPI Instance
- **ClockPolarity:** This parameter can be one of the following values:
  - LL\_SPI\_POLARITY\_LOW
  - LL\_SPI\_POLARITY\_HIGH

### Return values

- **None:**

### Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

### Reference Manual to LL API cross reference:

- CR1 CPOL LL\_SPI\_SetClockPolarity

## LL\_SPI\_GetClockPolarity

### Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetClockPolarity (SPI_TypeDef * SPIx)
```

### Function description

Get clock polarity.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_POLARITY\_LOW
  - LL\_SPI\_POLARITY\_HIGH

### Reference Manual to LL API cross reference:

- CR1 CPOL LL\_SPI\_GetClockPolarity

## LL\_SPI\_SetBaudRatePrescaler

### Function name

```
__STATIC_INLINE void LL_SPI_SetBaudRatePrescaler (SPI_TypeDef * SPIx, uint32_t BaudRate)
```

### Function description

Set baud rate prescaler.

## Parameters

- **SPIx:** SPI Instance
- **BaudRate:** This parameter can be one of the following values:
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV2
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV4
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV8
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV16
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV32
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV64
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV128
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV256

## Return values

- **None:**

## Notes

- These bits should not be changed when communication is ongoing. SPI BaudRate = fPCLK/Prescaler.

## Reference Manual to LL API cross reference:

- CR1 BR LL\_SPI\_SetBaudRatePrescaler

### LL\_SPI\_GetBaudRatePrescaler

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_SPI\_GetBaudRatePrescaler (SPI\_TypeDef \* SPIx)**

## Function description

Get baud rate prescaler.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV2
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV4
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV8
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV16
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV32
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV64
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV128
  - LL\_SPI\_BAUDRATEPRESCALER\_DIV256

## Reference Manual to LL API cross reference:

- CR1 BR LL\_SPI\_GetBaudRatePrescaler

### LL\_SPI\_SetTransferBitOrder

## Function name

**\_\_STATIC\_INLINE void LL\_SPI\_SetTransferBitOrder (SPI\_TypeDef \* SPIx, uint32\_t BitOrder)**

## Function description

Set transfer bit order.



## Parameters

- **SPIx:** SPI Instance
- **BitOrder:** This parameter can be one of the following values:
  - LL\_SPI\_LSB\_FIRST
  - LL\_SPI\_MSB\_FIRST

## Return values

- **None:**

## Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

## Reference Manual to LL API cross reference:

- CR1 LSBFIRST LL\_SPI\_SetTransferBitOrder

### LL\_SPI\_GetTransferBitOrder

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_SPI\_GetTransferBitOrder (SPI\_TypeDef \* SPIx)**

## Function description

Get transfer bit order.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_LSB\_FIRST
  - LL\_SPI\_MSB\_FIRST

## Reference Manual to LL API cross reference:

- CR1 LSBFIRST LL\_SPI\_GetTransferBitOrder

### LL\_SPI\_SetTransferDirection

## Function name

**\_\_STATIC\_INLINE void LL\_SPI\_SetTransferDirection (SPI\_TypeDef \* SPIx, uint32\_t TransferDirection)**

## Function description

Set transfer direction mode.

## Parameters

- **SPIx:** SPI Instance
- **TransferDirection:** This parameter can be one of the following values:
  - LL\_SPI\_FULL\_DUPLEX
  - LL\_SPI\_SIMPLEX\_RX
  - LL\_SPI\_HALF\_DUPLEX\_RX
  - LL\_SPI\_HALF\_DUPLEX\_TX

## Return values

- **None:**

## Notes

- For Half-Duplex mode, Rx Direction is set by default. In master mode, the MOSI pin is used and in slave mode, the MISO pin is used for Half-Duplex.

#### Reference Manual to LL API cross reference:

- CR1 RXONLY LL\_SPI\_SetTransferDirection
- CR1 BIDIMODE LL\_SPI\_SetTransferDirection
- CR1 BIDIOE LL\_SPI\_SetTransferDirection

#### LL\_SPI\_GetTransferDirection

##### Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetTransferDirection (SPI_TypeDef * SPIx)
```

##### Function description

Get transfer direction mode.

##### Parameters

- **SPIx:** SPI Instance

##### Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_FULL\_DUPLEX
  - LL\_SPI\_SIMPLEX\_RX
  - LL\_SPI\_HALF\_DUPLEX\_RX
  - LL\_SPI\_HALF\_DUPLEX\_TX

#### Reference Manual to LL API cross reference:

- CR1 RXONLY LL\_SPI\_GetTransferDirection
- CR1 BIDIMODE LL\_SPI\_GetTransferDirection
- CR1 BIDIOE LL\_SPI\_GetTransferDirection

#### LL\_SPI\_SetDataWidth

##### Function name

```
__STATIC_INLINE void LL_SPI_SetDataWidth (SPI_TypeDef * SPIx, uint32_t DataWidth)
```

##### Function description

Set frame data width.

##### Parameters

- **SPIx:** SPI Instance
- **DataWidth:** This parameter can be one of the following values:
  - LL\_SPI\_DATAWIDTH\_8BIT
  - LL\_SPI\_DATAWIDTH\_16BIT

##### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR1 DFF LL\_SPI\_SetDataWidth

#### LL\_SPI\_GetDataWidth

##### Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetDataWidth (SPI_TypeDef * SPIx)
```

##### Function description

Get frame data width.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_DATAWIDTH\_8BIT
  - LL\_SPI\_DATAWIDTH\_16BIT

## Reference Manual to LL API cross reference:

- CR1 DFF LL\_SPI\_GetDataWidth

## LL\_SPI\_EnableCRC

## Function name

```
__STATIC_INLINE void LL_SPI_EnableCRC (SPI_TypeDef * SPIx)
```

## Function description

Enable CRC.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **None:**

## Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

## Reference Manual to LL API cross reference:

- CR1 CRCEN LL\_SPI\_EnableCRC

## LL\_SPI\_DisableCRC

## Function name

```
__STATIC_INLINE void LL_SPI_DisableCRC (SPI_TypeDef * SPIx)
```

## Function description

Disable CRC.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **None:**

## Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

## Reference Manual to LL API cross reference:

- CR1 CRCEN LL\_SPI\_DisableCRC

## LL\_SPI\_IsEnabledCRC

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledCRC (SPI_TypeDef * SPIx)
```

## Function description

Check if CRC is enabled.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

## Reference Manual to LL API cross reference:

- CR1 CRCEN LL\_SPI\_IsEnabledCRC

## LL\_SPI\_SetCRCNext

## Function name

```
__STATIC_INLINE void LL_SPI_SetCRCNext (SPI_TypeDef * SPIx)
```

## Function description

Set CRCNext to transfer CRC on the line.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **None:**

## Notes

- This bit has to be written as soon as the last data is written in the SPIx\_DR register.

## Reference Manual to LL API cross reference:

- CR1 CRCNEXT LL\_SPI\_SetCRCNext

## LL\_SPI\_SetCRCPolynomial

## Function name

```
__STATIC_INLINE void LL_SPI_SetCRCPolynomial (SPI_TypeDef * SPIx, uint32_t CRCPoly)
```

## Function description

Set polynomial for CRC calculation.

## Parameters

- **SPIx:** SPI Instance
- **CRCPoly:** This parameter must be a number between Min\_Data = 0x00 and Max\_Data = 0xFFFF

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- CRCPOLY LL\_SPI\_SetCRCPolynomial

## LL\_SPI\_GetCRCPolynomial

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetCRCPolynomial (SPI_TypeDef * SPIx)
```

## Function description

Get polynomial for CRC calculation.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value is a number between Min\_Data = 0x00 and Max\_Data = 0xFFFF

## Reference Manual to LL API cross reference:

- CRCPR CRCPOLY LL\_SPI\_GetCRCPolynomial

## LL\_SPI\_GetRxCRC

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetRxCRC (SPI_TypeDef * SPIx)
```

## Function description

Get Rx CRC.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value is a number between Min\_Data = 0x00 and Max\_Data = 0xFFFF

## Reference Manual to LL API cross reference:

- RXCR CR RXCRC LL\_SPI\_GetRxCRC

## LL\_SPI\_GetTxCRC

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetTxCRC (SPI_TypeDef * SPIx)
```

## Function description

Get Tx CRC.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value is a number between Min\_Data = 0x00 and Max\_Data = 0xFFFF

## Reference Manual to LL API cross reference:

- TXCR CR TXCRC LL\_SPI\_GetTxCRC

## LL\_SPI\_SetNSSMode

## Function name

```
__STATIC_INLINE void LL_SPI_SetNSSMode (SPI_TypeDef * SPIx, uint32_t NSS)
```

## Function description

Set NSS mode.

## Parameters

- **SPIx:** SPI Instance
- **NSS:** This parameter can be one of the following values:
  - LL\_SPI\_NSS\_SOFT
  - LL\_SPI\_NSS\_HARD\_INPUT
  - LL\_SPI\_NSS\_HARD\_OUTPUT

## Return values

- **None:**

## Notes

- LL\_SPI\_NSS\_SOFT Mode is not used in SPI TI mode.

## Reference Manual to LL API cross reference:

- CR1 SSM LL\_SPI\_SetNSSMode
- 
- CR2 SSOE LL\_SPI\_SetNSSMode

### LL\_SPI\_GetNSSMode

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetNSSMode (SPI_TypeDef * SPIx)
```

## Function description

Get NSS mode.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_SPI\_NSS\_SOFT
  - LL\_SPI\_NSS\_HARD\_INPUT
  - LL\_SPI\_NSS\_HARD\_OUTPUT

## Reference Manual to LL API cross reference:

- CR1 SSM LL\_SPI\_GetNSSMode
- 
- CR2 SSOE LL\_SPI\_GetNSSMode

### LL\_SPI\_IsActiveFlag\_RXNE

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_RXNE (SPI_TypeDef * SPIx)
```

## Function description

Check if Rx buffer is not empty.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- SR RXNE LL\_SPI\_IsActiveFlag\_RXNE

## LL\_SPI\_IsActiveFlag\_TXE

### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_TXE (SPI_TypeDef * SPIx)
```

### Function description

Check if Tx buffer is empty.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- SR TXE LL\_SPI\_IsActiveFlag\_TXE

## LL\_SPI\_IsActiveFlag\_CRCERR

### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_CRCERR (SPI_TypeDef * SPIx)
```

### Function description

Get CRC error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- SR CRCERR LL\_SPI\_IsActiveFlag\_CRCERR

## LL\_SPI\_IsActiveFlag\_MODF

### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_MODF (SPI_TypeDef * SPIx)
```

### Function description

Get mode fault error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- SR MODF LL\_SPI\_IsActiveFlag\_MODF

## LL\_SPI\_IsActiveFlag\_OVR

### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_OVR (SPI_TypeDef * SPIx)
```

## Function description

Get overrun error flag.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- SR OVR LL\_SPI\_IsActiveFlag\_OVR

**LL\_SPI\_IsActiveFlag\_BSY**

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_BSY (SPI_TypeDef * SPIx)
```

## Function description

Get busy flag.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Notes

- The BSY flag is cleared under any one of the following conditions: -When the SPI is correctly disabled -When a fault is detected in Master mode (MODF bit set to 1) -In Master mode, when it finishes a data transmission and no new data is ready to be sent -In Slave mode, when the BSY flag is set to '0' for at least one SPI clock cycle between each data transfer.

## Reference Manual to LL API cross reference:

- SR BSY LL\_SPI\_IsActiveFlag\_BSY

**LL\_SPI\_IsActiveFlag\_FRE**

## Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_FRE (SPI_TypeDef * SPIx)
```

## Function description

Get frame format error flag.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- SR FRE LL\_SPI\_IsActiveFlag\_FRE

**LL\_SPI\_ClearFlag\_CRCERR**

## Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_CRCERR (SPI_TypeDef * SPIx)
```



### Function description

Clear CRC error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- SR CRCERR LL\_SPI\_ClearFlag\_CRCERR

### LL\_SPI\_ClearFlag\_MODF

### Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_MODF (SPI_TypeDef * SPIx)
```

### Function description

Clear mode fault error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Notes

- Clearing this flag is done by a read access to the SPIx\_SR register followed by a write access to the SPIx\_CR1 register

### Reference Manual to LL API cross reference:

- SR MODF LL\_SPI\_ClearFlag\_MODF

### LL\_SPI\_ClearFlag\_OVR

### Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_OVR (SPI_TypeDef * SPIx)
```

### Function description

Clear overrun error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Notes

- Clearing this flag is done by a read access to the SPIx\_DR register followed by a read access to the SPIx\_SR register

### Reference Manual to LL API cross reference:

- SR OVR LL\_SPI\_ClearFlag\_OVR

### LL\_SPI\_ClearFlag\_FRE

### Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_FRE (SPI_TypeDef * SPIx)
```

### Function description

Clear frame format error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Notes

- Clearing this flag is done by reading SPIx\_SR register

### Reference Manual to LL API cross reference:

- SR FRE LL\_SPI\_ClearFlag\_FRE

### LL\_SPI\_EnableIT\_ERR

### Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_ERR (SPI_TypeDef * SPIx)
```

### Function description

Enable error interrupt.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Notes

- This bit controls the generation of an interrupt when an error condition occurs (CRCERR, OVR, MODF in SPI mode, FRE at TI mode).

### Reference Manual to LL API cross reference:

- CR2\_ERRIE LL\_SPI\_EnableIT\_ERR

### LL\_SPI\_EnableIT\_RXNE

### Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_RXNE (SPI_TypeDef * SPIx)
```

### Function description

Enable Rx buffer not empty interrupt.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR2\_RXNEIE LL\_SPI\_EnableIT\_RXNE

### LL\_SPI\_EnableIT\_TXE

### Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_TXE (SPI_TypeDef * SPIx)
```

### Function description

Enable Tx buffer empty interrupt.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_SPI\_EnableIT\_TXE

### LL\_SPI\_DisableIT\_ERR

### Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_ERR (SPI_TypeDef * SPIx)
```

### Function description

Disable error interrupt.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Notes

- This bit controls the generation of an interrupt when an error condition occurs (CRCERR, OVR, MODF in SPI mode, FRE at TI mode).

### Reference Manual to LL API cross reference:

- CR2 ERRIE LL\_SPI\_DisableIT\_ERR

### LL\_SPI\_DisableIT\_RXNE

### Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_RXNE (SPI_TypeDef * SPIx)
```

### Function description

Disable Rx buffer not empty interrupt.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR2 RXNEIE LL\_SPI\_DisableIT\_RXNE

### LL\_SPI\_DisableIT\_TXE

### Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_TXE (SPI_TypeDef * SPIx)
```

### Function description

Disable Tx buffer empty interrupt.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_SPI\_DisableIT\_TXE

#### LL\_SPI\_IsEnabledIT\_ERR

#### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_ERR (SPI_TypeDef * SPIx)
```

#### Function description

Check if error interrupt is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 ERRIE LL\_SPI\_IsEnabledIT\_ERR

#### LL\_SPI\_IsEnabledIT\_RXNE

#### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_RXNE (SPI_TypeDef * SPIx)
```

#### Function description

Check if Rx buffer not empty interrupt is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 RXNEIE LL\_SPI\_IsEnabledIT\_RXNE

#### LL\_SPI\_IsEnabledIT\_TXE

#### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_TXE (SPI_TypeDef * SPIx)
```

#### Function description

Check if Tx buffer empty interrupt.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_SPI\_IsEnabledIT\_TXE

#### LL\_SPI\_EnableDMAReq\_RX

#### Function name

```
__STATIC_INLINE void LL_SPI_EnableDMAReq_RX (SPI_TypeDef * SPIx)
```

#### Function description

Enable DMA Rx.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_SPI\_EnableDMAReq\_RX

#### LL\_SPI\_DisableDMAReq\_RX

#### Function name

```
__STATIC_INLINE void LL_SPI_DisableDMAReq_RX (SPI_TypeDef * SPIx)
```

#### Function description

Disable DMA Rx.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_SPI\_DisableDMAReq\_RX

#### LL\_SPI\_IsEnabledDMAReq\_RX

#### Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledDMAReq_RX (SPI_TypeDef * SPIx)
```

#### Function description

Check if DMA Rx is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_SPI\_IsEnabledDMAReq\_RX

#### LL\_SPI\_EnableDMAReq\_TX

#### Function name

```
__STATIC_INLINE void LL_SPI_EnableDMAReq_TX (SPI_TypeDef * SPIx)
```

**Function description**

Enable DMA Tx.

**Parameters**

- **SPIx:** SPI Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- CR2 TXDMAEN LL\_SPI\_EnableDMAReq\_TX

**LL\_SPI\_DisableDMAReq\_TX**

**Function name**

**\_\_STATIC\_INLINE void LL\_SPI\_DisableDMAReq\_TX (SPI\_TypeDef \* SPIx)**

**Function description**

Disable DMA Tx.

**Parameters**

- **SPIx:** SPI Instance

**Return values**

- **None:**

**Reference Manual to LL API cross reference:**

- CR2 TXDMAEN LL\_SPI\_DisableDMAReq\_TX

**LL\_SPI\_IsEnabledDMAReq\_TX**

**Function name**

**\_\_STATIC\_INLINE uint32\_t LL\_SPI\_IsEnabledDMAReq\_TX (SPI\_TypeDef \* SPIx)**

**Function description**

Check if DMA Tx is enabled.

**Parameters**

- **SPIx:** SPI Instance

**Return values**

- **State:** of bit (1 or 0).

**Reference Manual to LL API cross reference:**

- CR2 TXDMAEN LL\_SPI\_IsEnabledDMAReq\_TX

**LL\_SPI\_DMA\_GetRegAddr**

**Function name**

**\_\_STATIC\_INLINE uint32\_t LL\_SPI\_DMA\_GetRegAddr (SPI\_TypeDef \* SPIx)**

**Function description**

Get the data register address used for DMA transfer.

**Parameters**

- **SPIx:** SPI Instance

#### Return values

- **Address:** of data register

#### Reference Manual to LL API cross reference:

- DR DR LL\_SPI\_DMA\_GetRegAddr

#### LL\_SPI\_ReceiveData8

#### Function name

```
__STATIC_INLINE uint8_t LL_SPI_ReceiveData8 (SPI_TypeDef * SPIx)
```

#### Function description

Read 8-Bits in the data register.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **RxData:** Value between Min\_Data=0x00 and Max\_Data=0xFF

#### Reference Manual to LL API cross reference:

- DR DR LL\_SPI\_ReceiveData8

#### LL\_SPI\_ReceiveData16

#### Function name

```
__STATIC_INLINE uint16_t LL_SPI_ReceiveData16 (SPI_TypeDef * SPIx)
```

#### Function description

Read 16-Bits in the data register.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **RxData:** Value between Min\_Data=0x00 and Max\_Data=0xFFFF

#### Reference Manual to LL API cross reference:

- DR DR LL\_SPI\_ReceiveData16

#### LL\_SPI\_TransmitData8

#### Function name

```
__STATIC_INLINE void LL_SPI_TransmitData8 (SPI_TypeDef * SPIx, uint8_t TxData)
```

#### Function description

Write 8-Bits in the data register.

#### Parameters

- **SPIx:** SPI Instance
- **TxData:** Value between Min\_Data=0x00 and Max\_Data=0xFF

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- DR DR LL\_SPI\_TransmitData8

## LL\_SPI\_TransmitData16

### Function name

```
__STATIC_INLINE void LL_SPI_TransmitData16 (SPI_TypeDef * SPIx, uint16_t TxData)
```

### Function description

Write 16-Bits in the data register.

### Parameters

- **SPIx:** SPI Instance
- **TxDat**a: Value between Min\_Data=0x00 and Max\_Data=0xFFFF

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- DR DR LL\_SPI\_TransmitData16

## LL\_SPI\_DeInit

### Function name

```
ErrorStatus LL_SPI_DeInit (SPI_TypeDef * SPIx)
```

### Function description

De-initialize the SPI registers to their default reset values.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: SPI registers are de-initialized
  - ERROR: SPI registers are not de-initialized

## LL\_SPI\_Init

### Function name

```
ErrorStatus LL_SPI_Init (SPI_TypeDef * SPIx, LL_SPI_InitTypeDef * SPI_InitStruct)
```

### Function description

Initialize the SPI registers according to the specified parameters in SPI\_InitStruct.

### Parameters

- **SPIx:** SPI Instance
- **SPI\_InitStruct:** pointer to a LL\_SPI\_InitTypeDef structure

### Return values

- **An:** ErrorStatus enumeration value. (Return always SUCCESS)

### Notes

- As some bits in SPI configuration registers can only be written when the SPI is disabled (SPI\_CR1\_SPE bit =0), SPI peripheral should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.



## LL\_SPI\_StructInit

### Function name

```
void LL_SPI_StructInit (LL_SPI_InitTypeDef * SPI_InitStruct)
```

### Function description

Set each LL\_SPI\_InitTypeDef field to default value.

### Parameters

- **SPI\_InitStruct:** pointer to a LL\_SPI\_InitTypeDef structure whose fields will be set to default values.

### Return values

- **None:**

## LL\_I2S\_Enable

### Function name

```
__STATIC_INLINE void LL_I2S_Enable (SPI_TypeDef * SPIx)
```

### Function description

Select I2S mode and Enable I2S peripheral.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- I2SCFGR I2SMOD LL\_I2S\_Enable
- I2SCFGR I2SE LL\_I2S\_Enable

## LL\_I2S\_Disable

### Function name

```
__STATIC_INLINE void LL_I2S_Disable (SPI_TypeDef * SPIx)
```

### Function description

Disable I2S peripheral.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- I2SCFGR I2SE LL\_I2S\_Disable

## LL\_I2S\_IsEnabled

### Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabled (SPI_TypeDef * SPIx)
```

### Function description

Check if I2S peripheral is enabled.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **State:** of bit (1 or 0).

## Reference Manual to LL API cross reference:

- I2SCFGR I2SE LL\_I2S\_IsEnabled

## LL\_I2S\_SetDataFormat

## Function name

```
__STATIC_INLINE void LL_I2S_SetDataFormat (SPI_TypeDef * SPIx, uint32_t DataFormat)
```

## Function description

Set I2S data frame length.

## Parameters

- **SPIx:** SPI Instance
- **DataFormat:** This parameter can be one of the following values:
  - LL\_I2S\_DATAFORMAT\_16B
  - LL\_I2S\_DATAFORMAT\_16B\_EXTENDED
  - LL\_I2S\_DATAFORMAT\_24B
  - LL\_I2S\_DATAFORMAT\_32B

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- I2SCFGR DATLEN LL\_I2S\_SetDataFormat
- I2SCFGR CHLEN LL\_I2S\_SetDataFormat

## LL\_I2S\_GetDataFormat

## Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetDataFormat (SPI_TypeDef * SPIx)
```

## Function description

Get I2S data frame length.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_I2S\_DATAFORMAT\_16B
  - LL\_I2S\_DATAFORMAT\_16B\_EXTENDED
  - LL\_I2S\_DATAFORMAT\_24B
  - LL\_I2S\_DATAFORMAT\_32B

## Reference Manual to LL API cross reference:

- I2SCFGR DATLEN LL\_I2S\_GetDataFormat
- I2SCFGR CHLEN LL\_I2S\_GetDataFormat

## LL\_I2S\_SetClockPolarity

### Function name

```
__STATIC_INLINE void LL_I2S_SetClockPolarity (SPI_TypeDef * SPIx, uint32_t ClockPolarity)
```

### Function description

Set I2S clock polarity.

### Parameters

- **SPIx:** SPI Instance
- **ClockPolarity:** This parameter can be one of the following values:
  - LL\_I2S\_POLARITY\_LOW
  - LL\_I2S\_POLARITY\_HIGH

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- I2SCFGR CKPOL LL\_I2S\_SetClockPolarity

## LL\_I2S\_GetClockPolarity

### Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetClockPolarity (SPI_TypeDef * SPIx)
```

### Function description

Get I2S clock polarity.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **Returned:** value can be one of the following values:
  - LL\_I2S\_POLARITY\_LOW
  - LL\_I2S\_POLARITY\_HIGH

### Reference Manual to LL API cross reference:

- I2SCFGR CKPOL LL\_I2S\_GetClockPolarity

## LL\_I2S\_SetStandard

### Function name

```
__STATIC_INLINE void LL_I2S_SetStandard (SPI_TypeDef * SPIx, uint32_t Standard)
```

### Function description

Set I2S standard protocol.

### Parameters

- **SPIx:** SPI Instance
- **Standard:** This parameter can be one of the following values:
  - LL\_I2S\_STANDARD\_PHILIPS
  - LL\_I2S\_STANDARD\_MSB
  - LL\_I2S\_STANDARD\_LSB
  - LL\_I2S\_STANDARD\_PCM\_SHORT
  - LL\_I2S\_STANDARD\_PCM\_LONG

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- I2SCFGR I2SSTD LL\_I2S\_SetStandard
- I2SCFGR PCMSYNC LL\_I2S\_SetStandard

#### LL\_I2S\_GetStandard

#### Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetStandard (SPI_TypeDef * SPIx)
```

#### Function description

Get I2S standard protocol.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **Returned:** value can be one of the following values:
  - LL\_I2S\_STANDARD\_PHILIPS
  - LL\_I2S\_STANDARD\_MSB
  - LL\_I2S\_STANDARD\_LSB
  - LL\_I2S\_STANDARD\_PCM\_SHORT
  - LL\_I2S\_STANDARD\_PCM\_LONG

#### Reference Manual to LL API cross reference:

- I2SCFGR I2SSTD LL\_I2S\_GetStandard
- I2SCFGR PCMSYNC LL\_I2S\_GetStandard

#### LL\_I2S\_SetTransferMode

#### Function name

```
__STATIC_INLINE void LL_I2S_SetTransferMode (SPI_TypeDef * SPIx, uint32_t Mode)
```

#### Function description

Set I2S transfer mode.

#### Parameters

- **SPIx:** SPI Instance
- **Mode:** This parameter can be one of the following values:
  - LL\_I2S\_MODE\_SLAVE\_TX
  - LL\_I2S\_MODE\_SLAVE\_RX
  - LL\_I2S\_MODE\_MASTER\_TX
  - LL\_I2S\_MODE\_MASTER\_RX

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- I2SCFGR I2SCFG LL\_I2S\_SetTransferMode

#### LL\_I2S\_GetTransferMode

#### Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetTransferMode (SPI_TypeDef * SPIx)
```

## Function description

Get I2S transfer mode.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_I2S\_MODE\_SLAVE\_TX
  - LL\_I2S\_MODE\_SLAVE\_RX
  - LL\_I2S\_MODE\_MASTER\_TX
  - LL\_I2S\_MODE\_MASTER\_RX

## Reference Manual to LL API cross reference:

- I2SCFGR I2SCFG LL\_I2S\_GetTransferMode

## LL\_I2S\_SetPrescalerLinear

## Function name

```
__STATIC_INLINE void LL_I2S_SetPrescalerLinear (SPI_TypeDef * SPIx, uint8_t PrescalerLinear)
```

## Function description

Set I2S linear prescaler.

## Parameters

- **SPIx:** SPI Instance
- **PrescalerLinear:** Value between Min\_Data=0x02 and Max\_Data=0xFF

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- I2SPR I2SDIV LL\_I2S\_SetPrescalerLinear

## LL\_I2S\_GetPrescalerLinear

## Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetPrescalerLinear (SPI_TypeDef * SPIx)
```

## Function description

Get I2S linear prescaler.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **PrescalerLinear:** Value between Min\_Data=0x02 and Max\_Data=0xFF

## Reference Manual to LL API cross reference:

- I2SPR I2SDIV LL\_I2S\_GetPrescalerLinear

## LL\_I2S\_SetPrescalerParity

## Function name

```
__STATIC_INLINE void LL_I2S_SetPrescalerParity (SPI_TypeDef * SPIx, uint32_t PrescalerParity)
```

## Function description

Set I2S parity prescaler.

## Parameters

- **SPIx:** SPI Instance
- **PrescalerParity:** This parameter can be one of the following values:
  - LL\_I2S\_PRESCALER\_PARITY\_EVEN
  - LL\_I2S\_PRESCALER\_PARITY\_ODD

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- I2SPR ODD LL\_I2S\_SetPrescalerParity

## LL\_I2S\_GetPrescalerParity

## Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_GetPrescalerParity (SPI\_TypeDef \* SPIx)**

## Function description

Get I2S parity prescaler.

## Parameters

- **SPIx:** SPI Instance

## Return values

- **Returned:** value can be one of the following values:
  - LL\_I2S\_PRESCALER\_PARITY\_EVEN
  - LL\_I2S\_PRESCALER\_PARITY\_ODD

## Reference Manual to LL API cross reference:

- I2SPR ODD LL\_I2S\_GetPrescalerParity

## LL\_I2S\_EnableMasterClock

## Function name

**\_\_STATIC\_INLINE void LL\_I2S\_EnableMasterClock (SPI\_TypeDef \* SPIx)**

## Function description

Enable the master clock output (Pin MCK)

## Parameters

- **SPIx:** SPI Instance

## Return values

- **None:**

## Reference Manual to LL API cross reference:

- I2SPR MCKOE LL\_I2S\_EnableMasterClock

## LL\_I2S\_DisableMasterClock

## Function name

**\_\_STATIC\_INLINE void LL\_I2S\_DisableMasterClock (SPI\_TypeDef \* SPIx)**

### Function description

Disable the master clock output (Pin MCK)

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- I2SPR MCKOE LL\_I2S\_DisableMasterClock

**LL\_I2S\_IsEnabledMasterClock**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_IsEnabledMasterClock (SPI\_TypeDef \* SPIx)**

### Function description

Check if the master clock output (Pin MCK) is enabled.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- I2SPR MCKOE LL\_I2S\_IsEnabledMasterClock

**LL\_I2S\_EnableAsyncStart**

### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_EnableAsyncStart (SPI\_TypeDef \* SPIx)**

### Function description

Enable asynchronous start.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- I2SCFGR ASTRTEN LL\_I2S\_EnableAsyncStart

**LL\_I2S\_DisableAsyncStart**

### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_DisableAsyncStart (SPI\_TypeDef \* SPIx)**

### Function description

Disable asynchronous start.

### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- I2SCFGR ASTRTEN LL\_I2S\_DisableAsyncStart

#### LL\_I2S\_IsEnabledAsyncStart

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_IsEnabledAsyncStart (SPI\_TypeDef \* SPIx)**

#### Function description

Check if asynchronous start is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- I2SCFGR ASTRTEN LL\_I2S\_IsEnabledAsyncStart

#### LL\_I2S\_IsActiveFlag\_RXNE

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_IsActiveFlag\_RXNE (SPI\_TypeDef \* SPIx)**

#### Function description

Check if Rx buffer is not empty.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- SR RXNE LL\_I2S\_IsActiveFlag\_RXNE

#### LL\_I2S\_IsActiveFlag\_TXE

#### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_IsActiveFlag\_TXE (SPI\_TypeDef \* SPIx)**

#### Function description

Check if Tx buffer is empty.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- SR TXE LL\_I2S\_IsActiveFlag\_TXE



**LL\_I2S\_IsActiveFlag\_BSY****Function name**

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_BSY (SPI_TypeDef * SPIx)
```

**Function description**

Get busy flag.

**Parameters**

- **SPIx**: SPI Instance

**Return values**

- **State**: of bit (1 or 0).

**Reference Manual to LL API cross reference:**

- SR BSY LL\_I2S\_IsActiveFlag\_BSY

**LL\_I2S\_IsActiveFlag\_OVR****Function name**

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_OVR (SPI_TypeDef * SPIx)
```

**Function description**

Get overrun error flag.

**Parameters**

- **SPIx**: SPI Instance

**Return values**

- **State**: of bit (1 or 0).

**Reference Manual to LL API cross reference:**

- SR OVR LL\_I2S\_IsActiveFlag\_OVR

**LL\_I2S\_IsActiveFlag\_UDR****Function name**

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_UDR (SPI_TypeDef * SPIx)
```

**Function description**

Get underrun error flag.

**Parameters**

- **SPIx**: SPI Instance

**Return values**

- **State**: of bit (1 or 0).

**Reference Manual to LL API cross reference:**

- SR UDR LL\_I2S\_IsActiveFlag\_UDR

**LL\_I2S\_IsActiveFlag\_FRE****Function name**

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_FRE (SPI_TypeDef * SPIx)
```

### Function description

Get frame format error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- SR FRE LL\_I2S\_IsActiveFlag\_FRE

**LL\_I2S\_IsActiveFlag\_CHSIDE**

### Function name

**\_\_STATIC\_INLINE uint32\_t LL\_I2S\_IsActiveFlag\_CHSIDE (SPI\_TypeDef \* SPIx)**

### Function description

Get channel side flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Notes

- 0: Channel Left has to be transmitted or has been received 1: Channel Right has to be transmitted or has been received It has no significance in PCM mode.

### Reference Manual to LL API cross reference:

- SR CHSIDE LL\_I2S\_IsActiveFlag\_CHSIDE

**LL\_I2S\_ClearFlag\_OVR**

### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_ClearFlag\_OVR (SPI\_TypeDef \* SPIx)**

### Function description

Clear overrun error flag.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- SR OVR LL\_I2S\_ClearFlag\_OVR

**LL\_I2S\_ClearFlag\_UDR**

### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_ClearFlag\_UDR (SPI\_TypeDef \* SPIx)**

### Function description

Clear underrun error flag.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- SR UDR LL\_I2S\_ClearFlag\_UDR

#### LL\_I2S\_ClearFlag\_FRE

#### Function name

```
__STATIC_INLINE void LL_I2S_ClearFlag_FRE (SPI_TypeDef * SPIx)
```

#### Function description

Clear frame format error flag.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- SR FRE LL\_I2S\_ClearFlag\_FRE

#### LL\_I2S\_EnableIT\_ERR

#### Function name

```
__STATIC_INLINE void LL_I2S_EnableIT_ERR (SPI_TypeDef * SPIx)
```

#### Function description

Enable error IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Notes

- This bit controls the generation of an interrupt when an error condition occurs (OVR, UDR and FRE in I2S mode).

#### Reference Manual to LL API cross reference:

- CR2 ERRIE LL\_I2S\_EnableIT\_ERR

#### LL\_I2S\_EnableIT\_RXNE

#### Function name

```
__STATIC_INLINE void LL_I2S_EnableIT_RXNE (SPI_TypeDef * SPIx)
```

#### Function description

Enable Rx buffer not empty IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 RXNEIE LL\_I2S\_EnableIT\_RXNE

**LL\_I2S\_EnableIT\_TXE**

#### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_EnableIT\_TXE (SPI\_TypeDef \* SPIx)**

#### Function description

Enable Tx buffer empty IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_I2S\_EnableIT\_TXE

**LL\_I2S\_DisableIT\_ERR**

#### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_DisableIT\_ERR (SPI\_TypeDef \* SPIx)**

#### Function description

Disable error IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Notes

- This bit controls the generation of an interrupt when an error condition occurs (OVR, UDR and FRE in I2S mode).

#### Reference Manual to LL API cross reference:

- CR2 ERRIE LL\_I2S\_DisableIT\_ERR

**LL\_I2S\_DisableIT\_RXNE**

#### Function name

**\_\_STATIC\_INLINE void LL\_I2S\_DisableIT\_RXNE (SPI\_TypeDef \* SPIx)**

#### Function description

Disable Rx buffer not empty IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 RXNEIE LL\_I2S\_DisableIT\_RXNE

#### LL\_I2S\_DisableIT\_TXE

#### Function name

```
__STATIC_INLINE void LL_I2S_DisableIT_TXE (SPI_TypeDef * SPIx)
```

#### Function description

Disable Tx buffer empty IT.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_I2S\_DisableIT\_TXE

#### LL\_I2S\_IsEnabledIT\_ERR

#### Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_ERR (SPI_TypeDef * SPIx)
```

#### Function description

Check if ERR IT is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 ERRIE LL\_I2S\_IsEnabledIT\_ERR

#### LL\_I2S\_IsEnabledIT\_RXNE

#### Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_RXNE (SPI_TypeDef * SPIx)
```

#### Function description

Check if RXNE IT is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 RXNEIE LL\_I2S\_IsEnabledIT\_RXNE

#### LL\_I2S\_IsEnabledIT\_TXE

#### Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_TXE (SPI_TypeDef * SPIx)
```

### Function description

Check if TXE IT is enabled.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **State:** of bit (1 or 0).

### Reference Manual to LL API cross reference:

- CR2 TXEIE LL\_I2S\_IsEnabledIT\_TXE

### LL\_I2S\_EnableDMAReq\_RX

### Function name

```
__STATIC_INLINE void LL_I2S_EnableDMAReq_RX (SPI_TypeDef * SPIx)
```

### Function description

Enable DMA Rx.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_I2S\_EnableDMAReq\_RX

### LL\_I2S\_DisableDMAReq\_RX

### Function name

```
__STATIC_INLINE void LL_I2S_DisableDMAReq_RX (SPI_TypeDef * SPIx)
```

### Function description

Disable DMA Rx.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_I2S\_DisableDMAReq\_RX

### LL\_I2S\_IsEnabledDMAReq\_RX

### Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledDMAReq_RX (SPI_TypeDef * SPIx)
```

### Function description

Check if DMA Rx is enabled.

### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL\_I2S\_IsEnabledDMAReq\_RX

LL\_I2S\_EnableDMAReq\_TX

#### Function name

`__STATIC_INLINE void LL_I2S_EnableDMAReq_TX (SPI_TypeDef * SPIx)`

#### Function description

Enable DMA Tx.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL\_I2S\_EnableDMAReq\_TX

LL\_I2S\_DisableDMAReq\_TX

#### Function name

`__STATIC_INLINE void LL_I2S_DisableDMAReq_TX (SPI_TypeDef * SPIx)`

#### Function description

Disable DMA Tx.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **None:**

#### Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL\_I2S\_DisableDMAReq\_TX

LL\_I2S\_IsEnabledDMAReq\_TX

#### Function name

`__STATIC_INLINE uint32_t LL_I2S_IsEnabledDMAReq_TX (SPI_TypeDef * SPIx)`

#### Function description

Check if DMA Tx is enabled.

#### Parameters

- **SPIx:** SPI Instance

#### Return values

- **State:** of bit (1 or 0).

#### Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL\_I2S\_IsEnabledDMAReq\_TX

## LL\_I2S\_ReceiveData16

### Function name

```
__STATIC_INLINE uint16_t LL_I2S_ReceiveData16 (SPI_TypeDef * SPIx)
```

### Function description

Read 16-Bits in data register.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **RxDData:** Value between Min\_Data=0x0000 and Max\_Data=0xFFFF

### Reference Manual to LL API cross reference:

- DR DR LL\_I2S\_ReceiveData16

## LL\_I2S\_TransmitData16

### Function name

```
__STATIC_INLINE void LL_I2S_TransmitData16 (SPI_TypeDef * SPIx, uint16_t TxData)
```

### Function description

Write 16-Bits in data register.

### Parameters

- **SPIx:** SPI Instance
- **TxDData:** Value between Min\_Data=0x0000 and Max\_Data=0xFFFF

### Return values

- **None:**

### Reference Manual to LL API cross reference:

- DR DR LL\_I2S\_TransmitData16

## LL\_I2S\_DeInit

### Function name

```
ErrorStatus LL_I2S_DeInit (SPI_TypeDef * SPIx)
```

### Function description

De-initialize the SPI/I2S registers to their default reset values.

### Parameters

- **SPIx:** SPI Instance

### Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: SPI registers are de-initialized
  - ERROR: SPI registers are not de-initialized

## LL\_I2S\_Init

### Function name

```
ErrorStatus LL_I2S_Init (SPI_TypeDef * SPIx, LL_I2S_InitTypeDef * I2S_InitStruct)
```



## Function description

Initializes the SPI/I2S registers according to the specified parameters in I2S\_InitStruct.

## Parameters

- **SPIx:** SPI Instance
- **I2S\_InitStruct:** pointer to a LL\_I2S\_InitTypeDef structure

## Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: SPI registers are Initialized
  - ERROR: SPI registers are not Initialized

## Notes

- As some bits in SPI configuration registers can only be written when the SPI is disabled (SPI\_CR1\_SPE bit =0), SPI peripheral should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.

### LL\_I2S\_StructInit

## Function name

**void LL\_I2S\_StructInit (LL\_I2S\_InitTypeDef \* I2S\_InitStruct)**

## Function description

Set each LL\_I2S\_InitTypeDef field to default value.

## Parameters

- **I2S\_InitStruct:** pointer to a LL\_I2S\_InitTypeDef structure whose fields will be set to default values.

## Return values

- **None:**

### LL\_I2S\_ConfigPrescaler

## Function name

**void LL\_I2S\_ConfigPrescaler (SPI\_TypeDef \* SPIx, uint32\_t PrescalerLinear, uint32\_t PrescalerParity)**

## Function description

Set linear and parity prescaler.

## Parameters

- **SPIx:** SPI Instance
- **PrescalerLinear:** value Min\_Data=0x02 and Max\_Data=0xFF.
- **PrescalerParity:** This parameter can be one of the following values:
  - LL\_I2S\_PRESCALER\_PARITY\_EVEN
  - LL\_I2S\_PRESCALER\_PARITY\_ODD

## Return values

- **None:**

## Notes

- To calculate value of PrescalerLinear(I2SDIV[7:0] bits) and PrescalerParity(ODD bit) Check Audio frequency table and formulas inside Reference Manual (SPI/I2S).

### LL\_I2S\_InitFullDuplex

## Function name

**ErrorStatus LL\_I2S\_InitFullDuplex (SPI\_TypeDef \* I2Sxext, LL\_I2S\_InitTypeDef \* I2S\_InitStruct)**

### Function description

Configures the full duplex mode for the I2Sx peripheral using its extension I2Sxext according to the specified parameters in the I2S\_InitStruct.

### Parameters

- **I2Sxext:** SPI Instance
- **I2S\_InitStruct:** pointer to a LL\_I2S\_InitTypeDef structure

### Return values

- **An:** ErrorStatus enumeration value:
  - SUCCESS: I2Sxext registers are Initialized
  - ERROR: I2Sxext registers are not Initialized

### Notes

- The structure pointed by I2S\_InitStruct parameter should be the same used for the master I2S peripheral. In this case, if the master is configured as transmitter, the slave will be receiver and vice versa. Or you can force a different mode by modifying the field I2S\_Mode to the value I2S\_SlaveRx or I2S\_SlaveTx independently of the master configuration.

## 90.3 SPI Firmware driver defines

The following section lists the various define and macros of the module.

### 90.3.1 SPI

SPI

#### ***Baud Rate Prescaler***

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV2**

BaudRate control equal to fPCLK/2

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV4**

BaudRate control equal to fPCLK/4

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV8**

BaudRate control equal to fPCLK/8

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV16**

BaudRate control equal to fPCLK/16

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV32**

BaudRate control equal to fPCLK/32

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV64**

BaudRate control equal to fPCLK/64

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV128**

BaudRate control equal to fPCLK/128

#### **LL\_SPI\_BAUDRATEPRESCALER\_DIV256**

BaudRate control equal to fPCLK/256

#### ***Transmission Bit Order***

#### **LL\_SPI\_LSB\_FIRST**

Data is transmitted/received with the LSB first

**LL\_SPI\_MSB\_FIRST**

Data is transmitted/received with the MSB first

**CRC Calculation****LL\_SPI\_CRCCALCULATION\_DISABLE**

CRC calculation disabled

**LL\_SPI\_CRCCALCULATION\_ENABLE**

CRC calculation enabled

**Datawidth****LL\_SPI\_DATAWIDTH\_8BIT**

Data length for SPI transfer: 8 bits

**LL\_SPI\_DATAWIDTH\_16BIT**

Data length for SPI transfer: 16 bits

**Get Flags Defines****LL\_SPI\_SR\_RXNE**

Rx buffer not empty flag

**LL\_SPI\_SR\_TXE**

Tx buffer empty flag

**LL\_SPI\_SR\_BSY**

Busy flag

**LL\_SPI\_SR\_CRCERR**

CRC error flag

**LL\_SPI\_SR\_MODF**

Mode fault flag

**LL\_SPI\_SR\_OVR**

Overrun flag

**LL\_SPI\_SR\_FRE**

TI mode frame format error flag

**IT Defines****LL\_SPI\_CR2\_RXNEIE**

Rx buffer not empty interrupt enable

**LL\_SPI\_CR2\_TXEIE**

Tx buffer empty interrupt enable

**LL\_SPI\_CR2\_ERRIE**

Error interrupt enable

**LL\_I2S\_CR2\_RXNEIE**

Rx buffer not empty interrupt enable

**LL\_I2S\_CR2\_TXEIE**

Tx buffer empty interrupt enable

**LL\_I2S\_CR2\_ERRIE**

Error interrupt enable

**Operation Mode****LL\_SPI\_MODE\_MASTER**

Master configuration

**LL\_SPI\_MODE\_SLAVE**

Slave configuration

**Slave Select Pin Mode****LL\_SPI\_NSS\_SOFT**

NSS managed internally. NSS pin not used and free

**LL\_SPI\_NSS\_HARD\_INPUT**

NSS pin used in Input. Only used in Master mode

**LL\_SPI\_NSS\_HARD\_OUTPUT**

NSS pin used in Output. Only used in Slave mode as chip select

**Clock Phase****LL\_SPI\_PHASE\_1EDGE**

First clock transition is the first data capture edge

**LL\_SPI\_PHASE\_2EDGE**

Second clock transition is the first data capture edge

**Clock Polarity****LL\_SPI\_POLARITY\_LOW**

Clock to 0 when idle

**LL\_SPI\_POLARITY\_HIGH**

Clock to 1 when idle

**Serial Protocol****LL\_SPI\_PROTOCOL\_MOTOROLA**

Motorola mode. Used as default value

**LL\_SPI\_PROTOCOL\_TI**

TI mode

**Transfer Mode****LL\_SPI\_FULL\_DUPLEX**

Full-Duplex mode. Rx and Tx transfer on 2 lines

**LL\_SPI\_SIMPLEX\_RX**

Simplex Rx mode. Rx transfer only on 1 line

**LL\_SPI\_HALF\_DUPLEX\_RX**

Half-Duplex Rx mode. Rx transfer on 1 line

## LL\_SPI\_HALF\_DUPLEX\_TX

Half-Duplex Tx mode. Tx transfer on 1 line

### *Common Write and read registers Macros*

## LL\_SPI\_WriteReg

#### **Description:**

- Write a value in SPI register.

#### **Parameters:**

- `__INSTANCE__`: SPI Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

#### **Return value:**

- None

## LL\_SPI\_ReadReg

#### **Description:**

- Read a value in SPI register.

#### **Parameters:**

- `__INSTANCE__`: SPI Instance
- `__REG__`: Register to be read

#### **Return value:**

- Register: value